

Abstract

This dissertation formalises and mechanises *type slicing* – a technique that incrementally explains program types by highlighting minimal program fragments. Using this, a programmer can select any sub-term of a program and incrementally explore complex types and concisely explain type errors. Crucially, this includes highlighting context *surrounding* the term, differing from typical type error highlighting which simply states a term has the wrong type, but not what about the surrounding contexts explains *why* it has this type.

I formalise this in a bidirectional, gradually-typed core calculus with explicit System F style polymorphism, products, and sum types. Additionally, a novel *context classification* judgement to formalise the notion of splitting type information into that derived from a selected sub-term and that coming from the surrounding context, which can be applied to arbitrary bidirectional type systems. Further, the theory extends formally to ill-typed programs with error marks building upon the marked lambda calculus of Zhao et al.

Acknowledgements

I thank my supervisor, Anil Madhavapeddy, for his guidance throughout this project. I am grateful to Cyrus Omar and the wider Hazel team for maintaining the Hazel language and for many helpful hints and pointers to relevant implementation points. Particular thanks go to Matthew Keenan, for discussions on the slicing of product types and especially for his detailed feedback on this dissertation as a whole, and to Thomas Porter, for discussions on the NP-hardness of slicing.

Mechanisation legend: ○ fully mechanised, ⊠ mechanised with postulates, △ not mechanised.

Contents

1	Introduction	7
1.1	Explanation by Example	8
2	Background	12
2.1	Bidirectional Types	12
2.2	Example: System F	13
2.3	Gradual Types	13
2.4	Lattice Background	14
2.5	Well-Foundedness, Monotonicity, and Fixed Points	15
3	The Core Calculus	17
3.1	○ Syntax & Relations	17
3.2	○ Lattice Properties	19
3.3	○ Typing Rules	21
3.4	○ Context Classification	23
3.5	○ Metatheory: Graduality & Unicity	26
4	Synthesis Slices	28
4.1	○ Synthesis Slices	28
4.2	○ Minimality	29
4.3	⊠ Existence of Minimal Slices	29
4.4	○ Sub-slices	30
4.5	○ Decompositions	31
4.6	○ Contribution Slices	33
5	Analysis Slices	34
5.1	○ Analysis Slices	34
5.2	○ Minimality	34
5.3	⊠ Existence of Minimal Slices	34
6	Error Marking & Type Error Debugging	35
6.1	○ Error Marking	35
6.2	○ Marked Context Classification	36
6.3	The Debugging Recipe by Example	37
7	Optimising Type Slice Calculation	40
7.1	○ Products of Synthesis Slices	40
7.2	○ Term Slices	41
7.3	○ Term-Minimality	41
7.4	⊠ Existence of Minimal Term-Minimal Slices	41
7.5	○ Calculating Term-Minimal Slices	43
7.6	△ Fixed Point Algorithm for Case Expressions	44
7.7	△ Calculating Analysis Slices	46

8	$\triangle$ Minimum-Size Slicing	47
8.1	$\triangle$ Definitions	47
8.2	$\triangle$ The reduction	47
8.3	$\triangle$ Correctness	48
8.4	$\triangle$ Extension to Minimal Synthesis Slices	49
9	Relation to Other Forms of Slicing	50
9.1	Classical Program Slicing	50
9.2	Galois Slicing	50
9.3	Type Error Slicing	51
10	Further Extensions & Applications	53
10.1	Further Extensions	53
10.2	Further Applications	53
11	Conclusions	56
	References	57
A	$\bigcirc$ Context Classification: Full Rule Set	60

1 Introduction

Programmers report that debugging and testing consumes 20–60% of their development time [4], with a small fraction of difficult bugs absorbing a disproportionate share of that effort [1]. In statically typed languages, many such bugs are caught statically at compile time by *type errors*.

These type errors are conventionally reported at a single location, yet most errors genuinely involve more than one source location [36]. The reported location is often imprecise in *two* directions: often including code that does *not* directly contribute to the error, yet at the same time misses regions of the surrounding context that *do* (for example, non-local type information sourced from bindings). As a result, existing highlighting systems show both too much and too little context to the programmer for any given error. The error itself states only that the term has a differing type that was *expected*, not *why* the system expected it.

This dissertation develops the theory of *type slicing*: a decomposable highlighting system that allows terms to be selected and the type explained in a *bidirectional* locally inferred type system. A programmer selects an expression and queries a part of the type, v , that they wish to explain. The explanation picks out a minimal portion of the program that suffices to justify the queried type.

A *synthesis slice* (section 4) of a selected term answers “which parts of the term caused it to have type v ?”; an *analysis slice* (section 5) of a selected term answers the complementary question, “which parts of the surrounding program enforced that this term have type v ?”.

Slices can be *refined*: refining a query, say from an entire function type to just its codomain, gives a smaller highlighted region for the refined query, supporting interactive exploration complex types, where the programmer can incrementally focus into sub-parts of the type that are of interest. For example, maybe only a small region of a large type involved in a type error is erroneous.

The type slicing machinery applies even beyond errors (section 6); it is possible to query any program regions responsible for any type information of interest, even well-typed code. Hence, type slicing is a tool to explain *any* types in a program.

The accompanying Hazel prototype for this work is available on the `type-slicing-v2` branch of the Hazel repository. The earlier prototype evaluated by Carroll [8] is separate, and is available on the `witnesses-type-slicing` branch.

1.1 Explanation by Example

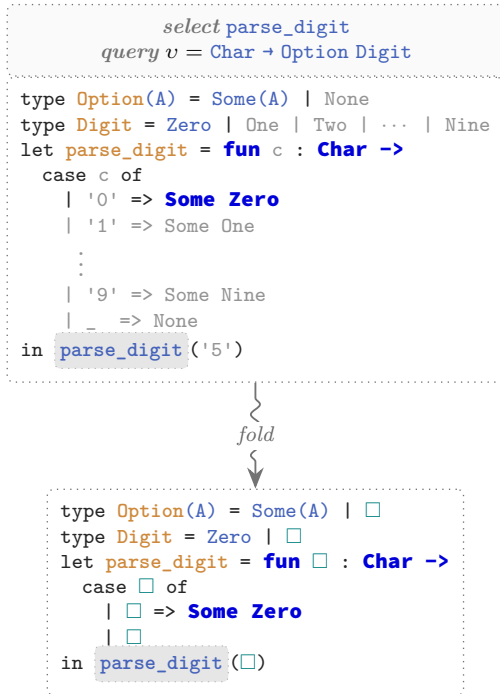
This section previews the slicing apparatus formally introduced in later chapters through three worked examples.

Figure 1 demonstrates a small example of a function parsing characters to single digits. In (a) the function is queried for its *synthesised* type, `Char → Option Digit`, which is explained by just taking a single branch from the function, and the corresponding part of the `Digit` type.

Secondly, (b) asks why the argument is *required* to be a `Char`, explained by an *analysis* slice retaining just the function application and the `Char` annotation on the function.

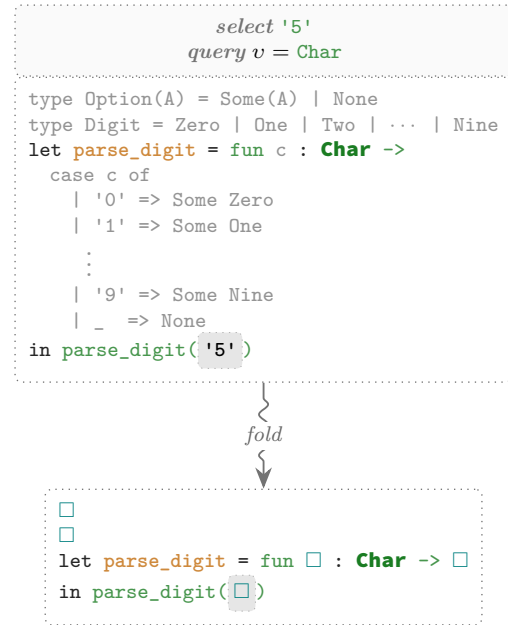
The formal mathematics of these two concepts are introduced in sections 4 and 5 explored upon a core calculus specified in section 3 which includes sum types, products, and explicit polymorphism.

Synthesis slice



(a) `parse_digit` synthesises type `Char → Option Digit`.

Analysis slice



(b) `'5'` analyses against type `Char`.

selected: The selected term
synthesis: Slice explaining why the selected term has queried type parts.
analysis: Slice explaining why the surrounding context enforces queried type parts.
 omitted: Omitted from slice (shown)
 □: Omitted from slice (folded)

structural¹: Required to form a valid program slice, but irrelevant to the type.
binding: Used assumptions whose bodies are recursively sliced.
bold/bold¹: Root sources of queried type information.

Figure 1: Parsing a character into an `Option Digit`.

¹ The distinction between purely structural parts of a slice and bolded type sources is not formalised, but is easy to implement in practice by tagging the leaves of the derivations during slicing.

These slices can be queried on *refined* types, investigating only a subpart of the full type. Figure 2 demonstrates the power of this by debugging a static type error: it defines a sequencing (`seq`) parser combinator to string together parsers sequentially, used to parse 4-digit pins.

- (1) The `seq` combinator is implemented incorrectly: the annotation and usage of the `f` argument forgets to take the remaining unparsed string `s'`. However, as this function is well-typed (but incorrect according to the programmers mental model), the error is localised to the passed argument at the usage of `seq`.
- (2) The synthesis slice explains the type of the term in the marked error.
- (2b) We can *refine* the synthesis slice to only show the *outermost inconsistency* in the type: that the argument provided is a curried function.

The sequence combinator should indeed take a two argument function, so this is correct. We instead look to the analysis slice for the error.

But, in general, if the error were inside the error squiggle, this slice shows that the squiggle highlights *more* than required to demonstrate the outermost error (the function itself, not the terms within).

- (2) The analysis slice explains the expected type, which is derived from the *incorrect annotation*.

The original error squiggle does not alert the programmer that the annotation itself could have been incorrect, despite this being a common occurrence (49% of the time [21]), at least when annotating existing code, which is common in hybrid static-dynamic typed language.

- (3b) The refined analysis slice refines the explanation to just the single argument function to *some* option type, which is what actually causes the conflict.
- (4) Because the slices do not *arbitrarily* blame the usage site like the error mark does, we inspect the refined analysis slice to locate and fix the error in the annotation.

After fixing the annotation, a secondary error at the application of `f` will be unveiled and correctly localised by the standard method.

This situation could arise naturally after refactoring the argument passed into `seq` to take an additional string argument. Here, the analysis slice points you directly to the corresponding annotation that must be changed to perform the refactor.

Section 6 concerns the formal theory applying type slicing to ill-typed programs with error marks, allowing for debugging of static type errors.

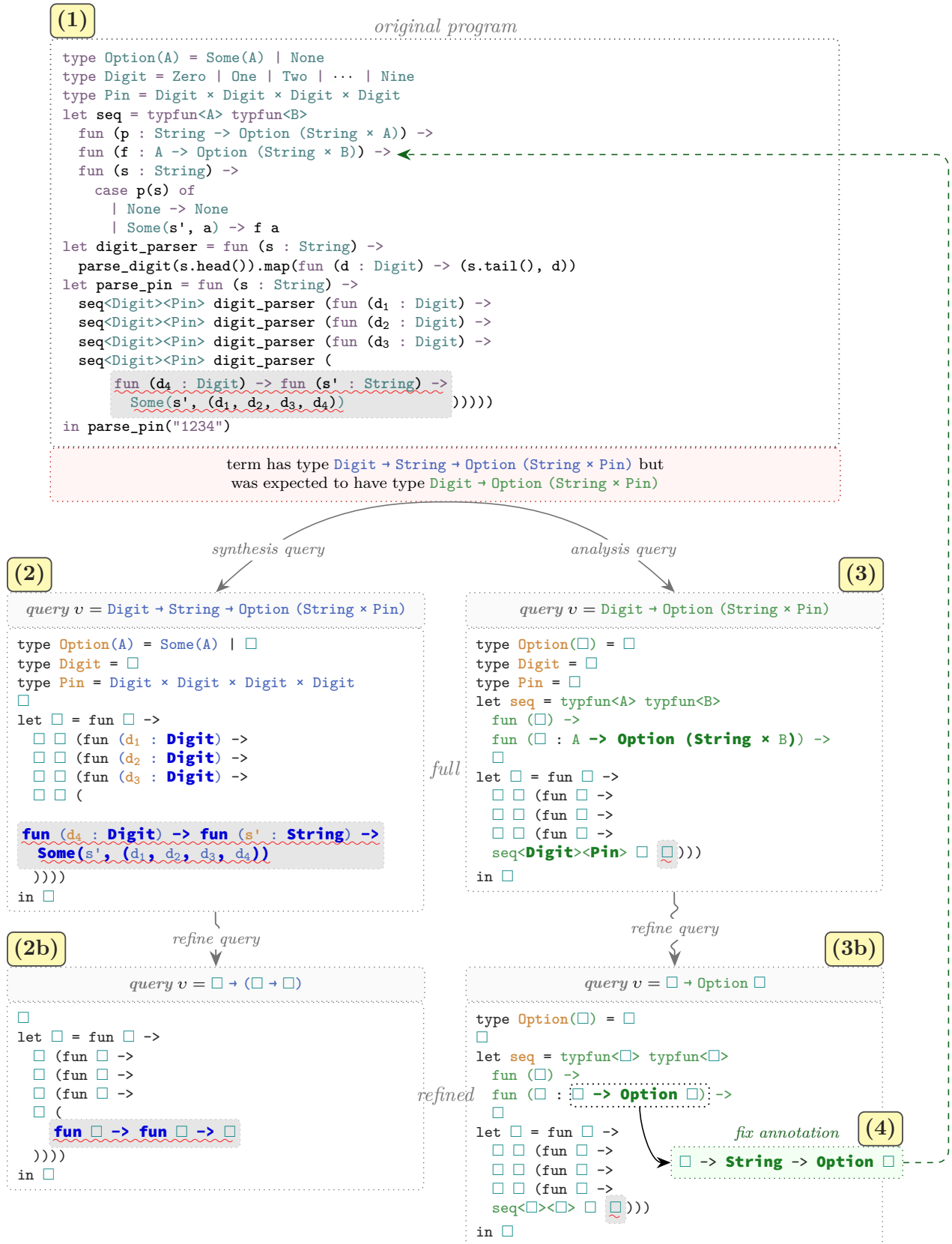


Figure 2: Debugging a static type error in a parser for 4 digit PINs.

Figure 3 demonstrates the power of type slicing when using *gradual types* by parsing a character to a digit, and otherwise falling back to a string, i.e. $\text{Char} \rightarrow \text{Digit} + \text{String}$. Using the dynamic type we can infer an unannotated injection $\text{Left}(a)$ into a sum type with unknown right injection $(A + \square)$ and $\text{Right}(b)$ at $\square + B$.

Therefore, in an if statement we can join two injections $(A + \square) \sqcup (\square + B)$ to determine a full static type $A + B$, with the slices identifying the branch from which each type was inferred. No sum type annotations were required. This behaviour is similar to union type inference, which is crucial in languages like TypeScript.

Importantly, we can use refined slices to query where each part of the sum type came from. In the example we find that the **Digit** came from the **then** branch, and the **String** from the right branch.

These situations are common in advanced type systems with set-theoretic types, where types are combined non-trivially by code branches. The example shows promise that type slicing could be especially applicable in explaining components of these set types by identifying the branches from which each type component is sourced. However, the current theory only explores combining dynamic code in branches and retrieving the corresponding static regions from the appropriate branches.

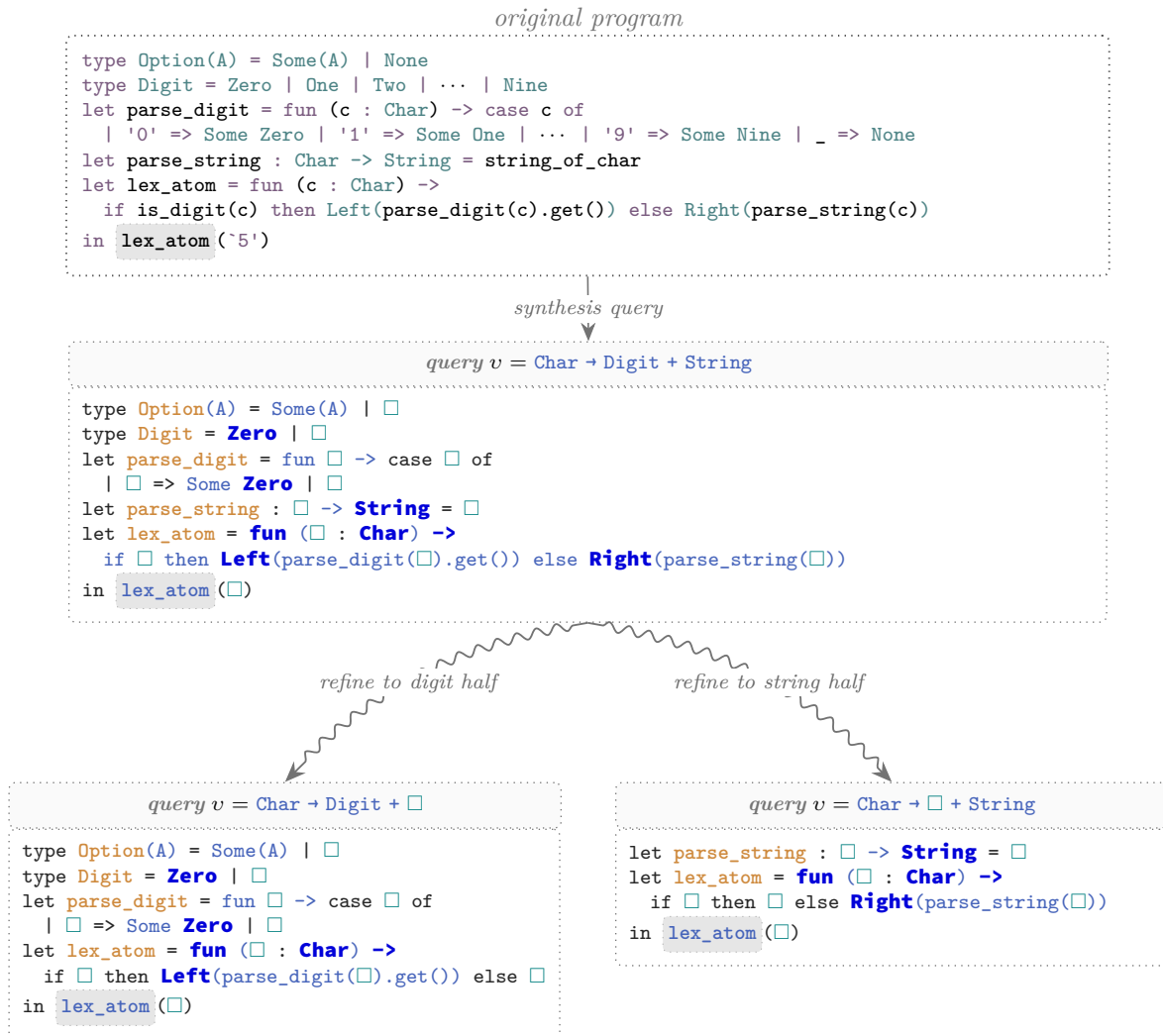


Figure 3: Lexing a character into a digit or string token.

2 Background

In this chapter I introduce the key concepts underlying the core calculus formalised: bidirectional typing, explicit polymorphism (System F), and gradual typing. I assume familiarity with standard type systems expressed via judgements, the simply-typed lambda calculus, and basic set notation.

2.1 Bidirectional Types

Type slicing is formalised for *bidirectional type systems* [14]. Such a type system is an algorithmic specification of typing judgements, naturally lending itself to direct typing algorithms which do not require *guessing* of types during type checking. These systems do require annotations, but annotation burden is lessened by locally inferring types [26] resulting in annotations only at redexes (for *most*² expressions).³

This is achieved by specifying the *mode* of the type parameter in a typing judgement, distinguishing when it is an *input* (type analysis/checking) and when it is an *output* (type synthesis):

$$\Delta; \Gamma \vdash e \Uparrow \tau$$

Read as: *e synthesises type τ under typing assumptions Γ . The type τ is an output.*

$$\Delta; \Gamma \vdash e \Downarrow \tau$$

Read as: *e analyses against type τ under typing assumptions Γ . The type τ is an input.* This is where the terminology of *synthesis* / *analysis* slices is derived from.

When designing such a system, care must be taken to ensure *mode correctness* [7]: an input must never need to be *guessed*. For example, the following function application rule is *not* mode correct:

$$\frac{\Delta; \Gamma \vdash e_1 \Downarrow \tau_1 \rightarrow \tau_2 \quad \Delta; \Gamma \vdash e_2 \Downarrow \tau_1}{\Delta; \Gamma \vdash e_1(e_2) \Downarrow \tau_2}$$

Here I try to check e_2 with input τ_1 , which is not known from either an output of any premise or from the input to the conclusion τ_2 . On the other hand, the following *is* mode correct:

$$\frac{\Delta; \Gamma \vdash e_1 \Uparrow \tau_1 \rightarrow \tau_2 \quad \Delta; \Gamma \vdash e_2 \Downarrow \tau_1}{\Delta; \Gamma \vdash e_1(e_2) \Uparrow \tau_2}$$

Now τ_1 is known, being synthesised from the first premise. We never have to “guess” any types.

Such languages have two fundamental rules:

$$\text{SVar} \frac{x : \tau \in \Gamma}{\Delta; \Gamma \vdash x \Uparrow \tau} \quad \text{Sub} \frac{\Delta; \Gamma \vdash e \Uparrow \tau}{\Delta; \Gamma \vdash e \Downarrow \tau}$$

Variables synthesise their type from the typing assumptions. Subsumption bridges the two modes: any term that can synthesise a type also analyses against that same type.

² Not always in the presence of binding elimination forms, e.g. case expressions. See section 4.5.4 of [14].

³ i.e. a function application must use an annotated function.

2.2 Example: System F

System F [25, ch. 23] extends the simply-typed lambda calculus with *parametric polymorphism*: type functions $\Lambda\alpha. e$, universal types $\forall\alpha. \tau$, and type application $e \langle \sigma \rangle$. A polymorphic function like the identity $\Lambda\alpha. \lambda x : \alpha. x$ has type $\forall\alpha. \alpha \Rightarrow \alpha$ and can be instantiated at any type.

System F can be expression within the bidirectional framework:

$$\text{S}\Lambda \frac{\Delta; \Gamma \vdash e \uparrow \tau}{\Delta; \Gamma \vdash \Lambda\alpha. e \uparrow \forall\alpha. \tau} \quad \text{STApp} \frac{\Delta; \Gamma \vdash e \uparrow \forall\alpha. \tau'}{\Delta; \Gamma \vdash e \langle \sigma \rangle \uparrow [0 \mapsto \sigma], \tau'}$$

Type functions synthesis a $\forall$ -type when the body synthesises under an extended type variable context which may now reference the newly bound type variable. Type application substitutes the bound type variable for a provided type argument.

2.3 Gradual Types

A *gradual type system* [30, 31] combines static and dynamic typing. Terms may be annotated with the dynamic type $\square$, marking regions of code omitted from static type-checking but still *interoperable* with statically typed code. For example, the following type-checks:

```
let x :  $\square$  = 10 in x ^ "str"
```

Where $\wedge$ is string concatenation expecting inputs to be `String`. This would then cause a runtime *cast error* when attempting to calculate $10 \wedge \text{"str"}$.

2.3.1 Consistency

This is enabled by a *consistency* relation $\tau_1 \sim \tau_2$, a weakening of type equality. Consistency replaces equality in the typing rules, allowing types to be used interchangeably when they are consistent. Every type is consistent with the dynamic type $\square$, and compound types are consistent when their sub-parts are:

$$\frac{}{\tau \sim \square} \quad \frac{}{\square \sim \tau} \quad \frac{}{\tau \sim \tau} \quad \frac{\tau_1 \sim \tau'_1 \quad \tau_2 \sim \tau'_2}{\tau_1 \rightarrow \tau_2 \sim \tau'_1 \rightarrow \tau'_2}$$

However, consistency is *not transitive*: $\text{Int} \sim \square \sim \text{Bool}$ does not imply $\text{Int} \sim \text{Bool}$. This non-transitivity is essential – preventing the dynamic type from collapsing all type distinctions.

After type-checking, explicit *casts* are inserted into partially annotated programs by elaboration. Then, at runtime, type errors which were masked by dynamic-type annotations are caught as *cast errors*.

2.3.2 Graduality

A true gradual type system satisfies a *gradual guarantee* property, first specified by Siek et al. in their refined criteria [31]. To specify this we first need to introduce the notion of *type precision*, written $\tau \sqsubseteq \tau'$, which orders types by their dynamic content: $\square$ is the least precise type, and concrete types like $\text{Int} \rightarrow \text{Bool}$ are maximally precise, while compound types can be partially dynamic, e.g. $\square \Rightarrow \text{Bool}$.

The only relevant part of graduality to Type Slicing is the downwards direction of the static gradual guarantee.

Downwards graduality. If a well-typed program is made *less precise* (type annotations replaced by $\square$), it remains well-typed, possibly with a less precise type. In a bidirectional setting: if $\Delta; \Gamma \vdash e \uparrow \tau$ and $\Gamma' \sqsubseteq \Gamma, e' \sqsubseteq e$, then $\exists \tau'. \Gamma' \vdash e' \uparrow \tau'$ with $\tau' \sqsubseteq \tau$.

2.4 Lattice Background

The foundations for program highlighting in Type Slicing are built upon lattices, which I introduce here.

A *partial order* $(L, \sqsubseteq)$ is a set L equipped with a reflexive, transitive, and antisymmetric relation $\sqsubseteq$. For example, precision in gradual typing forms a partial order, with $a \sqsubseteq b$ intuitively meaning “ b carries at least as much type information as a ”.

A *lattice* is a partial order in which every pair of elements in L have both a greatest lower bound (a *meet*) and a least upper bound (a *join*) under the partial order. A lattice is *bounded* if it has a bottom element $\perp$ ($\perp \sqsubseteq x$ for all x) and a top element $\top$ ($x \sqsubseteq \top$ for all x). A *meet-semilattice* is a partial order in which every pair has a meet but not necessarily a join; dually for a *join-semilattice*. A bounded lattice is *distributive* if $a \sqcap (b \sqcup c) = (a \sqcap b) \sqcup (a \sqcap c)$ for all a, b, c .

The meet and join operations have natural interpretations under the precision interpretation:

- The *meet* $a \sqcap b$ is the most precise element that agrees with both a and b , retaining the type information they share (Figure 4a).
- The *join* $a \sqcup b$ is the least precise element that is still as precise as both a and b , combining all the type information present in either (Figure 4b).

Heyting algebras. A bounded distributive lattice is a *Heyting algebra* [5, 13] if it has an *implication* operation $a \rightarrow b$ satisfying the adjunction:

$$c \sqcap a \sqsubseteq b \quad \text{iff} \quad c \sqsubseteq (a \rightarrow b)$$

That is, $a \rightarrow b = \bigsqcup \{c \mid c \sqcap a \sqsubseteq b\}$ is the largest element whose meet with a is below b . The *pseudocomplement* of a is $\neg a = a \rightarrow \perp$. Again, when interpreting $\sqsubseteq$ as type precision, $a \rightarrow b$ intuitively means: “what is the most precise element such that, combined with a , the result is no more precise than b ?” (Figure 4c.)

Co-Heyting algebras. Dually, a bounded distributive lattice is a *co-Heyting algebra* [27, 13] if it has a *subtraction* operation $a \setminus b$ satisfying:

$$a \sqsubseteq b \sqcup c \quad \text{iff} \quad a \setminus b \sqsubseteq c$$

i.e. $a \setminus b = \bigsqcap \{c \mid a \sqsubseteq b \sqcup c\}$ is the smallest element whose join with b is above a ; the *supplement* of a is $\sim a = \top \setminus a$. Reading $\sqsubseteq$ as type precision, $a \setminus b$ answers: “what type information in a is missing from b ?” (Figure 4d.)

Bi-Heyting algebras. A *bi-Heyting algebra* [27, 28] is simultaneously a Heyting algebra and a co-Heyting algebra. Every *finite* bounded distributive lattice is a bi-Heyting algebra, because the required suprema and infima are over finite sets [13].

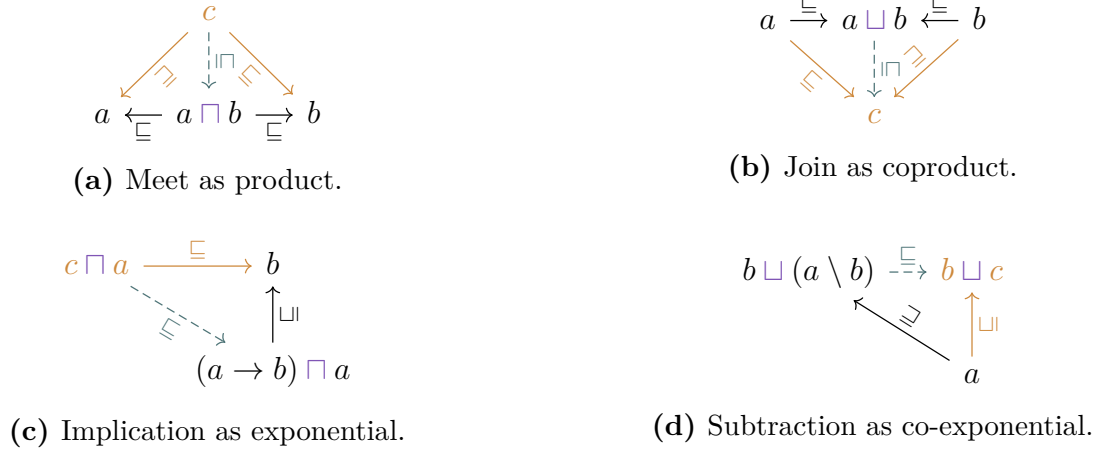


Figure 4: Lattice operations. Where **assuming** some relations (solid arrows) **induces** additional relations (dashed arrows).

Product lattices. Given two lattices $(L_1, \sqsubseteq_1)$ and $(L_2, \sqsubseteq_2)$, their *product* $L_1 \times L_2$ is a lattice defined componentwise: $(a_1, a_2) \sqsubseteq (b_1, b_2)$ iff $a_1 \sqsubseteq_1 b_1$ and $a_2 \sqsubseteq_2 b_2$ (Figure 5). Meets and joins are similarly defined componentwise. If both lattices are bounded, distributive, or bi-Heyting, so is the product.

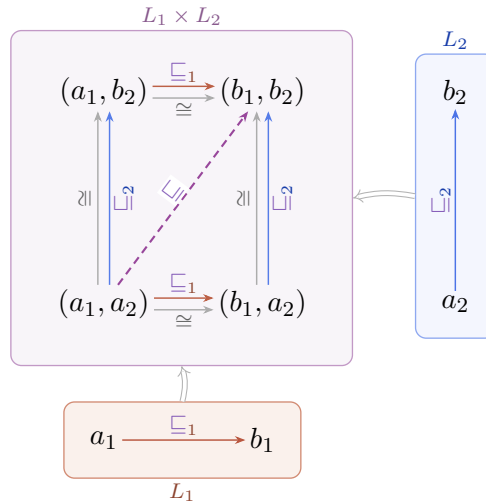


Figure 5: Product Lattice Precision

2.5 Well-Foundedness, Monotonicity, and Fixed Points

A partial order $(L, \sqsubseteq)$ is *well-founded* if it admits no infinite *strictly* descending chain $x_0 \sqsupset x_1 \sqsupset x_2 \sqsupset \dots$. This is immediate on finite sets: every chain is bounded by the finite cardinality $|L|$.

A function $f : L \rightarrow L$ is *monotone* if $x \sqsubseteq y$ implies $f(x) \sqsubseteq f(y)$, and *anti-monotone* if $x \sqsubseteq y$ implies $f(y) \sqsubseteq f(x)$.

Lemma 2.1 (Monotonicity of co-Heyting subtraction). *In a co-Heyting algebra $(L, \sqsubseteq, \sqcup, \setminus)$, the binary operation $\setminus$ is monotone in its first argument and anti-monotone in its second: for all $a, a', b, b' \in L$,*

$$a \sqsubseteq a' \Rightarrow a \setminus b \sqsubseteq a' \setminus b, \quad b \sqsubseteq b' \Rightarrow a \setminus b' \sqsubseteq a \setminus b.$$

Proof. By the universal property of $\setminus$, $x \setminus y \sqsubseteq z \iff x \sqsubseteq y \sqcup z$. *First argument:* take $z = a' \setminus b$. Then $a' \sqsubseteq b \sqcup z$, so $a \sqsubseteq a' \sqsubseteq b \sqcup z$, giving $a \setminus b \sqsubseteq z = a' \setminus b$. *Second argument:* take $z = a \setminus b$. Then $a \sqsubseteq b \sqcup z \sqsubseteq b' \sqcup z$, giving $a \setminus b' \sqsubseteq z = a \setminus b$. $\square$

Theorem 2.2 (Kleene Fixed Point, finite case). *Let $(L, \sqsubseteq, \perp)$ be a finite partial order with bottom element $\perp$, and let $f : L \rightarrow L$ be monotone. Then f has a least fixed point $\mu f = f^n(\perp)$ for some $n \leq |L|$.*

Proof. The chain $\perp \sqsubseteq f(\perp) \sqsubseteq f^2(\perp) \sqsubseteq \dots$ is ascending by monotonicity. Since L is finite, this chain must stabilise: there is some $n \leq |L|$ with $f^n(\perp) = f^{n+1}(\perp)$. So $f^n(\perp)$ is a fixed point. Further, $f^n(\perp)$ is a least fixed point: let x be any fixed point of f . We show $f^k(\perp) \sqsubseteq x$ by induction on k . Base: $\perp \sqsubseteq x$ since $\perp$ is the bottom. Step: $f^{k+1}(\perp) = f(f^k(\perp)) \sqsubseteq f(x) = x$ by monotonicity of f and the induction hypothesis. $\square$

$$\perp \xrightarrow{\sqsubseteq} f(\perp) \xrightarrow{\sqsubseteq} f^2(\perp) \longrightarrow \dots \xrightarrow{=} \mu f$$

Figure 6: Kleene iteration to the least fixed point μf .

3 The Core Calculus

Legend: $\bigcirc$ fully mechanised, $\bigodot$ mechanised with postulates, $\triangle$ not mechanised, $\square$ important theorem, $\square$ counterexample.

In this chapter I present the core calculus which Type Slicing builds upon. It is a bidirectional, gradually typed lambda calculus based on the Hazel calculus [20], extended with products, sums, and explicit System F polymorphism. Then, upon this, I define and prove several bi-Heyting lattices under a precision relation.

Core/*Base

3.1 $\bigcirc$ Syntax & Relations

The syntax of the core calculus is given in Figure 7. $*$ is the unit type, and $\square$ is a gap / hole serving a dual role: in the gradual typing interpretation it is the dynamic type, and in the slicing interpretation it represents unrequired type information (to explain a query). Similarly a $\square$ in expressions represents omitted sub-expressions, irrelevant to the explanation resulting from a query. In type checking, omitted expressions synthesise the dynamic (omitted) type.

Syntax Types τ and expressions e

$$\begin{aligned} \tau &::= \square \mid * \mid \tau \Rightarrow \tau \mid \tau \times \tau \mid \tau + \tau \mid \forall \alpha. \tau \mid \alpha \\ e &::= \square \mid * \mid x \mid \lambda x : \tau. e \mid \lambda x. e \mid e(e) \mid (e, e) \mid \pi_1 e \mid \pi_2 e \\ &\quad \mid \iota_1 e \mid \iota_2 e \mid (\text{case } e \text{ of } x. e \mid y. e) \mid \Lambda \alpha. e \mid e\langle \tau \rangle \mid \text{let } x = e \text{ in } e \end{aligned}$$

Derived forms Definable in terms of the core syntax

$$\begin{aligned} (e : \tau) &\triangleq (\lambda x : \tau. x)(e) & \text{Bool} &\triangleq * + * \\ \text{true} &\triangleq \iota_1 * & \text{false} &\triangleq \iota_2 * \\ \text{if } e \text{ then } e_1 \text{ else } e_2 &\triangleq (\text{case } e \text{ of } x. e_1 \mid y. e_2) & (x \notin \text{FV}(e_1), y \notin \text{FV}(e_2)) \end{aligned}$$

Figure 7: Syntax of the core calculus up to α -equivalence (on variables x). $\text{FV}(\cdot)$ denotes the set of free variables of an expression.

Core/Typ/
Consistency

3.1.1 $\bigcirc$ Consistency

Type consistency (Figure 8) is exactly as presented in standard gradual type systems (section 2.3). That is, a weakening of equality, where every type is consistent with $\square$, and compound types are consistent when their components are. Consistency is reflexive and symmetric but *not* transitive.

$\tau_1 \sim \tau_2$ τ_1 is consistent with τ_2

$$\begin{aligned} &\sim \square_1 \frac{}{\tau \sim \square} \quad \sim \square_2 \frac{}{\square \sim \tau} \quad \sim * \frac{}{* \sim *} \quad \sim \Rightarrow \frac{\tau_1 \sim \tau'_1 \quad \tau_2 \sim \tau'_2}{\tau_1 \Rightarrow \tau_2 \sim \tau'_1 \Rightarrow \tau'_2} \\ &\sim \times \frac{\tau_1 \sim \tau'_1 \quad \tau_2 \sim \tau'_2}{\tau_1 \times \tau_2 \sim \tau'_1 \times \tau'_2} \quad \sim + \frac{\tau_1 \sim \tau'_1 \quad \tau_2 \sim \tau'_2}{\tau_1 + \tau_2 \sim \tau'_1 + \tau'_2} \quad \sim \forall \frac{\tau \sim \tau'}{\forall \alpha. \tau \sim \forall \alpha. \tau'} \end{aligned}$$

Figure 8: Type consistency.

3.1.2 ○ Precision

Core/*/Precision

Precision (Figure 9) is a partial order that organises terms by *information content*: for types, $\tau' \sqsubseteq \tau$ means τ has at least as much type information as τ' . Here, $\square$ is the global minimum. Expression precision is defined analogously. Figure 10 illustrates this with a small example.

$\tau \sqsubseteq \tau'$ τ is less precise than τ'

$$\begin{array}{c}
 \frac{}{\square \sqsubseteq \tau} \quad \frac{}{* \sqsubseteq *} \quad \frac{}{\alpha \sqsubseteq \alpha} \quad \frac{}{\Rightarrow \sqsubseteq \Rightarrow} \frac{\tau_1 \sqsubseteq \tau'_1 \quad \tau_2 \sqsubseteq \tau'_2}{\tau_1 \Rightarrow \tau_2 \sqsubseteq \tau'_1 \Rightarrow \tau'_2} \\
 \frac{}{\times \sqsubseteq \times} \frac{\tau_1 \sqsubseteq \tau'_1 \quad \tau_2 \sqsubseteq \tau'_2}{\tau_1 \times \tau_2 \sqsubseteq \tau'_1 \times \tau'_2} \quad \frac{}{+ \sqsubseteq +} \frac{\tau_1 \sqsubseteq \tau'_1 \quad \tau_2 \sqsubseteq \tau'_2}{\tau_1 + \tau_2 \sqsubseteq \tau'_1 + \tau'_2} \quad \frac{}{\forall \sqsubseteq \forall} \frac{\tau \sqsubseteq \tau'}{\forall \alpha. \tau \sqsubseteq \forall \alpha. \tau'}
 \end{array}$$

Figure 9: Type precision.

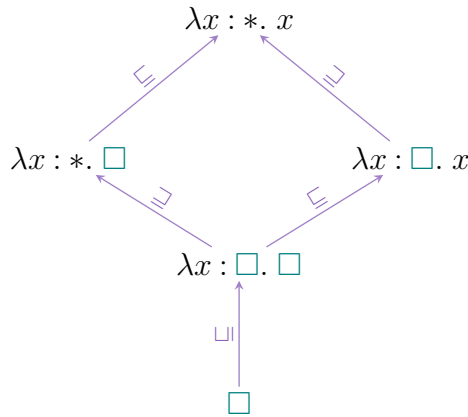


Figure 10: Hasse diagram of slices less precise than $\lambda x : *. x$, with each edge directed from a less-precise slice up to a more-precise slice.

Core/Assms/

Precision on Typing Assumptions. In this calculus, I represent typing assumptions as *finite* partial functions from variable names to types, with domain $\text{dom}(\Gamma)$. Precision is defined by domain inclusion and pointwise precision on types in the (smaller) domain:

$$\gamma_1 \sqsubseteq \gamma_2 \iff \text{dom}(\gamma_1) \subseteq \text{dom}(\gamma_2) \text{ and } \gamma_1(x) \sqsubseteq \gamma_2(x) \text{ for all } x \in \text{dom}(\gamma_1)$$

That is, a less precise type assumptions set either assumes fewer variables' types, or assumes less precise types on the present variables.

Remark 3.1 (Assumptions vs Contexts Terminology). Typical literature refers to these *typing assumptions* as 'typing contexts'. I avoid this terminology as I use the *context* to refer to the syntactic context around a subterm, i.e. in the same sense as used in contextual dynamics (Harper [16, Chapter 5]).

Strictly descending on successively less precise terms will always terminate at an 'empty' term in finite time. This is a useful property for calculating type slices (theorem 4.6).

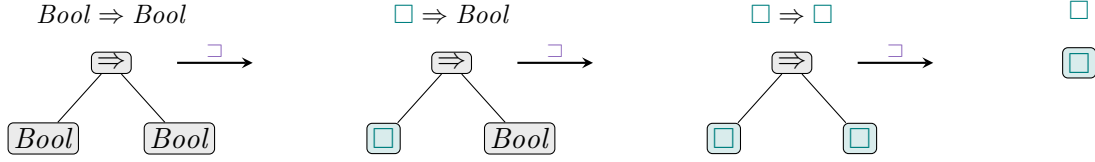


Figure 11: A maximal descending chain in $[Bool \Rightarrow Bool]$.

Proposition 3.2 ($\triangle$ Downward Well-Foundedness of Precision). *The strict precision ordering $\sqsubset$ is downward well-founded on types, expressions, and assumptions. Each family has a bottom element under precision:*

- *Types and expressions:* $\perp = \square$.
- *Assumptions:* $\perp = \emptyset$.

Proof. Each object is a finite tree over a finite alphabet of constructors. A strict descent in precision must replace at least one non- $\square$ position with $\square$, as illustrated in Figure 11. Since there are finitely many positions, any descending chain has length bounded by the number of positions in the original term. And for assumptions, the finiteness of $\text{dom}(\Gamma)$ bounds the number of variables available. $\square$

Core/*/Lattice

3.2 $\bigcirc$ Lattice Properties

3.2.1 $\bigcirc$ Meets and Joins

The *meet* $\tau_1 \sqcap \tau_2$ is the greatest lower bound (GLB) of two types under precision. It is defined inductively: types of the same ‘kind’⁴ take meets of their components, and $\square$ is an *annihilator* ($\square \sqcap \tau = \square$). The meet is *always* defined: when the outermost kinds differ, $\tau_1 \sqcap \tau_2 = \square$ is a valid (and greatest) lower bound. Since all pairs have a meet, we have that types form a *meet-semilattice* with lower bound $\square$ (Section 2.4). Expression meet is defined analogously. Assumption meets are defined pointwise and by intersecting the domains.

The *join* $\tau_1 \sqcup \tau_2$ is the least upper bound (LUB). It is similarly defined inductively and componentwise, but with $\square$ acting as the *identity* ($\square \sqcup \tau = \tau$, $\tau \sqcup \square = \tau$). However, unlike meets, the join is only defined for consistent types (i.e. types of the same kind): when τ_1 and τ_2 have different kinds, no upper bound exists.

Remark 3.3. One might hope to quotient types by consistency and form a join-semilattice. But, this fails because consistency is *not transitive*: $Int \sim \square \sim Bool$ does not imply $Int \sim Bool$.

Core/Instances

3.2.2 $\bigcirc$ The Slice Lattice

For a fixed type τ , the *slice lattice* of τ is the *lower set* under precision:

$$[\tau] = \{v \mid v \sqsubseteq \tau\}$$

$[e]$, $[\mathcal{C}]$, and $[\Gamma]$ are defined analogously for expressions, contexts, and assumptions. Each slice lattice has top $\top = \tau$ and bottom $\perp = \square$. Then precision, meets, and joins are simply lift unchanged into the slice lattice. Importantly all terms in the slice lattice are consistent, so joins are always defined:

⁴ Top level type constructor

Lemma 3.4 (○ Slices are Mutually Consistent). *If $v_1 \sqsubseteq \tau$ and $v_2 \sqsubseteq \tau$, then $v_1 \sim v_2$.*

Hence, these slice lattices are *bounded lattice*. We also get that the lattice is *distributive*. And, they are on finite lattices:

Theorem 3.5 (△ Slice Lattices are Finite). *Each slice lattice $\lfloor \tau \rfloor$, $\lfloor e \rfloor$, $\lfloor \mathcal{C} \rfloor$, and $\lfloor \Gamma \rfloor$ is finite. For a term with n positions, the slice lattice has at most 2^n elements.*

Every finite distributive lattice is a bi-Heyting algebra [13], so the slice lattices are bi-Heyting: in a finite lattice the implication and subtraction operations below can be defined directly as a finite join / meet, respectively.

Core/*/Lattice

Theorem 3.6 (○ Slice Lattices are Bounded Distributive Bi-Heyting Algebras). *Each slice lattice $\lfloor \tau \rfloor$, $\lfloor e \rfloor$, $\lfloor \mathcal{C} \rfloor$, and $\lfloor \Gamma \rfloor$ is a bounded distributive bi-Heyting algebra:*

- ○ **Bounded:** bottom $\perp = \square$ (or $\emptyset$ for assumptions), top $\top = \tau$ (or $e, \mathcal{C}, \Gamma$).
- ○ **Distributive:** meet distributes over join, $v_1 \sqcap (v_2 \sqcup v_3) = (v_1 \sqcap v_2) \sqcup (v_1 \sqcap v_3)$ (and dually).
- **Bi-Heyting:** equipped with
 - ○ **Implication:** $v_1 \rightarrow v_2 = \bigsqcup \{ v \mid v \sqcap v_1 \sqsubseteq v_2 \}$ – the most precise slice compatible with refining v_1 to at most v_2 ;
 - ○ **Subtraction:** $v_1 \setminus v_2 = \bigsqcap \{ v \mid v_1 \sqsubseteq v_2 \sqcup v \}$ – the least precise slice whose join with v_2 recovers v_1 .

Inductive Definition: Of course, calculating those joins / meets directly is very inefficient. We can define both inductively on slice lattices of the sub-terms, as demonstrated by example below for the lattice $\lfloor \tau_1 \Rightarrow \tau_2 \rfloor$ (inductively on $\lfloor \tau_1 \rfloor$ and $\lfloor \tau_2 \rfloor$).

- *Implication* $\cdot \rightarrow \cdot$

$$\begin{aligned}
 \square \rightarrow v &= \top \\
 v \rightarrow \square &= \square & (\text{when } v \sqsubseteq \square) \\
 * \rightarrow * &= * \\
 (v_1 \Rightarrow v_2) \rightarrow (v'_1 \Rightarrow v'_2) &= (v_1 \rightarrow v'_1) \Rightarrow (v_2 \rightarrow v'_2)
 \end{aligned}$$

- *Subtraction* $\cdot \setminus \cdot$

$$\begin{aligned}
 \square \setminus v &= \square \\
 v \setminus \square &= v \\
 * \setminus * &= \square \\
 (v_1 \Rightarrow v_2) \setminus (v'_1 \Rightarrow v'_2) &= \begin{cases} \square & \text{if } v_1 \setminus v'_1 = \square \text{ and } v_2 \setminus v'_2 = \square \\ (v_1 \setminus v'_1) \Rightarrow (v_2 \setminus v'_2) & \text{otherwise} \end{cases}
 \end{aligned}$$

Remark 3.7 (Mechanisation of Bi-Heyting operators). The implication and subtraction are only fully mechanised for the type-slice lattice $\lfloor \tau \rfloor$, which is the only lattice for which they are actually *used* in the forthcoming calculi.

Notation: Slice Lattices as Highlighted Terms. An element of a slice lattice can be notated as the original (top) term with omitted regions (replaced by a $\square$) left unhighlighted and kept regions highlighted. For instance, the slice $Int \Rightarrow \square$ in $[Int \Rightarrow Bool]$ can be represented as $Int \Rightarrow \square Bool$.

The lattice operations have direct visual interpretations with this notation: meet $\sqcap$ is the intersection of highlighted regions, join $\sqcup$ is the union, subtraction $\setminus$ keeps the source's (LHS) highlights that are absent from the target (RHS), and implication $\rightarrow$ keeps positions outside the source's highlights and those inside the target's. For example, in $[(*) \Rightarrow (*) \times (*)]$:

$$\begin{array}{lcl}
 (* \Rightarrow *) \times * \sqcap (* \Rightarrow *) \times * & = & (* \Rightarrow *) \times * \\
 (* \Rightarrow *) \times * \sqcup (* \Rightarrow *) \times * & = & (* \Rightarrow *) \times * \\
 (* \Rightarrow *) \times * \rightarrow (* \Rightarrow *) \times * & = & (* \Rightarrow *) \times * \\
 (* \Rightarrow *) \times * \setminus (* \Rightarrow *) \times * & = & (* \Rightarrow *) \times *
 \end{array}$$

No Boolean Lattice. The pseudo-complement $\neg v = v \rightarrow \square$ and the supplement $\sim v = \top \setminus v$ do *not* coincide on slice lattices, so they are not Boolean algebras. Take $v = (*) \Rightarrow *$ in $[(*) \Rightarrow (*)]$:

$$\begin{array}{lcl}
 \neg v & = & (*) \Rightarrow * \rightarrow * \Rightarrow * = * \Rightarrow *, \\
 \sim v & = & (*) \Rightarrow * \setminus (*) \Rightarrow * = * \Rightarrow *.
 \end{array}$$

The two disagree ($(*) \Rightarrow * \neq (*) \Rightarrow *$). Consequentially, the Boolean complementation laws fail on either side: the Law of the Excluded Middle fails for pseudo-negation, and the Principle of Contradiction fails for the supplement:

$$\begin{array}{lcl}
 v \sqcup \neg v & = & (*) \Rightarrow * \sqcup * \Rightarrow * = * \Rightarrow * \neq (*) \Rightarrow * \\
 v \sqcap \sim v & = & (*) \Rightarrow * \sqcap * \Rightarrow * = * \Rightarrow * \neq * \Rightarrow *.
 \end{array}$$

3.2.3 $\triangle$ Well-Foundedness of Slice Lattices

Proposition 3.8 ($\triangle$ Upward Well-Foundedness of Slice Lattices). *For each of the four families, the reversed strict precision ordering $\sqsupseteq$ on $[\cdot]$ is well-founded.*

Proof. Each slice lattice is finite (Theorem 3.5). A finite poset with a top element admits no infinite ascending chains. $\square$

Combined with downwards well-foundedness (Proposition 3.2), this gives well-foundedness in both directions within each slice.

3.3 $\bigcirc$ Typing Rules

The complete bidirectional typing rules are given in Figures 12 and 13, where types may contain variables which must be within a type variable context Δ . That is, a type τ is *well-formed* under Δ , written $\Delta \vdash \tau \text{ wf}$, if every free type variable in τ is a member of Δ .

The rules are mostly standard, but I highlight notable features:

$\boxed{\Delta; \Gamma \vdash e \Uparrow \tau}$ e synthesises type τ under type variable context Δ and typing assumptions Γ

$$\begin{array}{c}
\Uparrow\Box \frac{}{\Delta; \Gamma \vdash \Box \Uparrow \Box} \quad \Uparrow* \frac{}{\Delta; \Gamma \vdash * \Uparrow *} \\
\Uparrow\text{Var} \frac{x : \tau \in \Gamma}{\Delta; \Gamma \vdash x \Uparrow \tau} \\
\Uparrow\lambda\text{Ann} \frac{\Delta \vdash \tau_1 \text{ wf} \quad \Delta; \Gamma, x : \tau_1 \vdash e \Uparrow \tau_2}{\Delta; \Gamma \vdash \lambda x : \tau_1. e \Uparrow \tau_1 \Rightarrow \tau_2} \quad \Uparrow\text{let} \frac{\Delta; \Gamma \vdash e_1 \Uparrow \tau_1 \quad \Delta; \Gamma, x : \tau_1 \vdash e_2 \Uparrow \tau_2}{\Delta; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \Uparrow \tau_2} \\
\Uparrow\Lambda \frac{\Delta, \alpha; \Gamma \vdash e \Uparrow \tau}{\Delta; \Gamma \vdash \Lambda \alpha. e \Uparrow \forall \alpha. \tau} \quad \Uparrow\text{TApp} \frac{\Delta \vdash \sigma \text{ wf} \quad \Delta; \Gamma \vdash e \Uparrow \tau \quad \tau \sqcup \forall \alpha. \Box \equiv \forall \alpha. \tau'}{\Delta; \Gamma \vdash e \langle \sigma \rangle \Uparrow [\alpha \mapsto \sigma] \tau'} \\
\Uparrow\text{App} \frac{\Delta; \Gamma \vdash e_1 \Uparrow \tau \quad \tau \sqcup (\Box \Rightarrow \Box) \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash e_2 \Downarrow \tau_1}{\Delta; \Gamma \vdash e_1(e_2) \Uparrow \tau_2} \\
\Uparrow\& \frac{\Delta; \Gamma \vdash e_1 \Uparrow \tau_1 \quad \Delta; \Gamma \vdash e_2 \Uparrow \tau_2}{\Delta; \Gamma \vdash (e_1, e_2) \Uparrow \tau_1 \times \tau_2} \\
\Uparrow\pi_1 \frac{\Delta; \Gamma \vdash e \Uparrow \tau \quad \tau \sqcup (\Box \times \Box) \equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_1 e \Uparrow \tau_1} \quad \Uparrow\pi_2 \frac{\Delta; \Gamma \vdash e \Uparrow \tau \quad \tau \sqcup (\Box \times \Box) \equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_2 e \Uparrow \tau_2} \\
\Uparrow\iota_1 \frac{\Delta; \Gamma \vdash e \Uparrow \tau}{\Delta; \Gamma \vdash \iota_1 e \Uparrow \tau + \Box} \quad \Uparrow\iota_2 \frac{\Delta; \Gamma \vdash e \Uparrow \tau}{\Delta; \Gamma \vdash \iota_2 e \Uparrow \Box + \tau} \\
\Uparrow\text{case} \frac{\Delta; \Gamma \vdash e \Uparrow \tau \quad \tau \sqcup (\Box + \Box) \equiv \tau_1 + \tau_2 \quad \tau'_1 \sim \tau'_2 \quad \Delta; \Gamma, x : \tau_1 \vdash e_1 \Uparrow \tau'_1 \quad \Delta; \Gamma, y : \tau_2 \vdash e_2 \Uparrow \tau'_2}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e_1 \mid y. e_2 \Uparrow \tau'_1 \sqcup \tau'_2}
\end{array}$$

Figure 12: Synthesis rules.

$\boxed{\Delta; \Gamma \vdash e \Downarrow \tau}$ e analyses against type τ under Δ and Γ

$$\begin{array}{c}
\Downarrow\text{Sub} \frac{\Delta; \Gamma \vdash e \Uparrow \tau' \quad \tau \sim \tau'}{\Delta; \Gamma \vdash e \Downarrow \tau} \\
\Downarrow\lambda \frac{\tau \sqcup (\Box \Rightarrow \Box) \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma, x : \tau_1 \vdash e \Downarrow \tau_2}{\Delta; \Gamma \vdash \lambda x. e \Downarrow \tau} \\
\Downarrow\lambda\text{Ann} \frac{\Delta \vdash \tau_1 \text{ wf} \quad \tau \sim \tau_1 \Rightarrow \Box \quad \tau \sqcup \tau_1 \Rightarrow \Box \equiv \tau'_1 \Rightarrow \tau_2 \quad \Delta; \Gamma, x : \tau_1 \vdash e \Downarrow \tau_2}{\Delta; \Gamma \vdash \lambda x : \tau_1. e \Downarrow \tau} \\
\Downarrow\text{case} \frac{\Delta; \Gamma \vdash e \Uparrow \tau' \quad \tau' \sqcup (\Box + \Box) \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x : \tau_1 \vdash e_1 \Downarrow \tau \quad \Delta; \Gamma, y : \tau_2 \vdash e_2 \Downarrow \tau}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e_1 \mid y. e_2 \Downarrow \tau} \\
\Downarrow\iota_1 \frac{\tau \sqcup (\Box + \Box) \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash e \Downarrow \tau_1}{\Delta; \Gamma \vdash \iota_1 e \Downarrow \tau} \quad \Downarrow\iota_2 \frac{\tau \sqcup (\Box + \Box) \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash e \Downarrow \tau_2}{\Delta; \Gamma \vdash \iota_2 e \Downarrow \tau} \\
\Downarrow\& \frac{\tau \sqcup (\Box \times \Box) \equiv \tau_1 \times \tau_2 \quad \Delta; \Gamma \vdash e_1 \Downarrow \tau_1 \quad \Delta; \Gamma \vdash e_2 \Downarrow \tau_2}{\Delta; \Gamma \vdash (e_1, e_2) \Downarrow \tau} \\
\Downarrow\text{let} \frac{\Delta; \Gamma \vdash e_1 \Uparrow \tau_1 \quad \Delta; \Gamma, x : \tau_1 \vdash e_2 \Downarrow \tau}{\Delta; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \Downarrow \tau}
\end{array}$$

Figure 13: Analysis rules. $\Downarrow\text{Sub}$ is the unique bridge from analysis to synthesis.

- **Join-based matching:** Terms can synthesise $\square$ which is not syntactically a function, but should be treated as such (it is consistent with the function type). We can extract the argument and return types with a join: $\tau \sqcup (\square \Rightarrow \square) \equiv \tau_1 \Rightarrow \tau_2$. Similarly for other compound types.⁵
- **Case expressions** can be type-checked in synthesis mode ($\Uparrow$ case), require the branch types τ_1 and τ_2 to be *consistent*, synthesising the *join* $\tau_1 \sqcup \tau_2$.
- **Dynamic sum types:** Sum injections have synthesis rules which infer the absent component of the sum with $\square$: $\iota_1 e$ synthesises $\tau + \square$ where e synthesises τ (and dually for ι_2). This allows sum types to be inferred across case branches by joining types without annotations, like the behaviour shown in fig. 3 (section 1.1).
- **Annotated vs. unannotated lambdas:** Annotated lambdas, $\lambda x : \tau_1. e$, can synthesise their type using the annotation on the argument, while $\lambda x. e$ can only analyse as there is no way to determine the type of the argument at the definition site.
- **Unannotated Let bindings:** Let bindings, **let** $x = e_1$ **in** e_2 , *synthesise* e_1 and use this to type the body, without requiring annotation.

3.4 $\bigcirc$ Context Classification

Context classification is a novel construction for bidirectional type systems. It is inspired by one-hole contexts (zippers) of algebraic data types [17], decomposing a data structure into a focused element and its surrounding context. The same idea applies to *typing derivations*: given a derivation of an expression e and focusing on a sub-term e' in e , the classification decomposes it into the *surrounding context* (carrying all typing information *outside* the focused sub-term e') and the *focused sub-derivation* (carrying all typing information from *inside* the focused sub-term e').

The classification also records the extra assumptions made available at the focus by additional bound variables introduced in the captured context, and the mode of the focus (whether or not the sub-term synthesises a type). Both of these are necessary to allow type checking of the focus directly from the context classification derivation.

Remark 3.9. This construction replaces and generalises the notion of *checking contexts* from the previous type slicing theory.

3.4.1 $\bigcirc$ Context Syntax

First, I define one-hole contexts on terms in Figure 14. The focus of the context is notated $\bigcirc$ and is restricted to a single position in an expression.

3.4.2 $\bigcirc$ Context Precision and Plugging Contexts

Then, for any context, you can substitute a term into the focus to produce a complete term. This is notated by $\mathcal{C}[e]$, plugging e into context $\mathcal{C}$.

Precision on contexts, $\mathcal{C}' \sqsubseteq \mathcal{C}$, requires the two contexts to share the focus in the same structural position, with componentwise precision on sub-terms and sub-contexts. Within

⁵ This is equivalent to the standard use of ‘matching functions’ in the gradual typing literature, typically notated $\tau \blacktriangleright \tau_1 \Rightarrow \tau_2$.

Syntax One-hole expression contexts $\mathcal{C}$

$$\begin{aligned} \mathcal{C} ::= & \bigcirc \mid \lambda x : \tau. \mathcal{C} \mid \lambda x. \mathcal{C} \mid \mathcal{C}(e) \mid e(\mathcal{C}) \mid \mathcal{C}\langle\tau\rangle \\ & \mid (\mathcal{C}, e) \mid (e, \mathcal{C}) \mid \iota_1 \mathcal{C} \mid \iota_2 \mathcal{C} \mid \pi_1 \mathcal{C} \mid \pi_2 \mathcal{C} \\ & \mid \text{case } e \text{ of } x. \mathcal{C} \mid y. f \mid \text{case } e \text{ of } x. f \mid y. \mathcal{C} \\ & \mid \Lambda \alpha. \mathcal{C} \mid \text{let } x = \mathcal{C} \text{ in } e \mid \text{let } x = e \text{ in } \mathcal{C} \end{aligned}$$

$\mathcal{C}[e]$ Plug expression e into the hole $\bigcirc$

$$\begin{aligned} \bigcirc[e] &= e \\ (\lambda x : \tau. \mathcal{C})[e] &= \lambda x : \tau. \mathcal{C}[e] \\ (\mathcal{C}_1(e_2))[e] &= \mathcal{C}_1[e](e_2) \quad (\text{similarly for all other constructors}) \end{aligned}$$

$\square_{\mathcal{C}}$ Purely structural context: replace all sub-terms with $\square$, preserving structure

$$\square_{\bigcirc} = \bigcirc \quad \square_{\lambda x : \tau. \mathcal{C}} = \lambda x : \square. \square_{\mathcal{C}} \quad \square_{\mathcal{C}(e)} = \square_{\mathcal{C}}(\square) \quad \text{etc.}$$

Figure 14: One-hole expression contexts. $\bigcirc$ marks the hole; $\mathcal{C}[e]$ plugs it.

$$\begin{array}{ccc} (\mathcal{C}, e) & \xrightarrow{\text{plug}} & \mathcal{C}[e] \\ \uparrow \square & & \uparrow \square \\ (\mathcal{C}', e') & \xrightarrow{\text{plug}} & \mathcal{C}'[e'] \end{array}$$

Figure 15: Plug preserves precision (Lemma 3.10).

each slice lattice $\lfloor \mathcal{C} \rfloor$ (Section 3.2.2), the bottom is the *purely structural context* $\square_{\mathcal{C}}$: all sub-terms omitted, preserving only the focus location (Figure 14).

Downwards well-foundedness holds within each $\lfloor \mathcal{C} \rfloor$ by the same argument as Proposition 3.2.

Importantly, context plugging preserves precision:

Lemma 3.10 ($\bigcirc$ Plug Preserves Precision). *If $\mathcal{C}' \sqsubseteq \mathcal{C}$ and $e' \sqsubseteq e$, then $\mathcal{C}'[e'] \sqsubseteq \mathcal{C}[e]$. (Figure 15)*

This allows slicing to independently slice the *external* type information from the context and the *internal* type information of the focused term independently, while ensure the whole program slice (plugging the two together) remains a valid slice of the original program.

3.4.3 $\bigcirc$ The Classification Judgement

The context classification judgement is notated:

$$\Delta; \Gamma \vdash \mathcal{C} \text{ at } d \triangleright \Gamma' [m]$$

reading as: “under type variable context Δ and assumptions Γ , context $\mathcal{C}$ in a derivation of outer mode d has its focus typed under $\Delta'; \Gamma'$ in focus mode m ”.

The *outer derivation mode* d records whether the whole expression $\mathcal{C}[e]$ is in a synthesis derivation producing type τ ($d = \Uparrow \tau$) or an analysis derivation against τ ($d = \Downarrow \tau$). The *focus mode* m records what the focus e must satisfy: either $m = \Uparrow \tau'$ (the focus must synthesise type τ') or $m = \Downarrow \tau'$ (the focus must analyse against τ'). Both the outer derivation mode and focus mode carry a type, enabling the classification to track type flow through the derivation.

Further, we record the *focus assumptions* Γ' and the *focus mode* m as mentioned previously. Finally, several rule require *extra premises* which systematically match with the premises of the typing rule that each context classification rule represents.

Example. Consider the derivation of $\Delta; \Gamma \vdash e_1(e_2) \Uparrow \tau_2$ via $\Uparrow\text{App}$, where $\Delta; \Gamma \vdash e_1 \Uparrow \tau$, $\tau \sqcup (\Box \Rightarrow \Box) \equiv \tau_1 \Rightarrow \tau_2$, and $\Delta; \Gamma \vdash e_2 \Downarrow \tau_1$.

- **Function position** (e_1 is the focus, $\mathcal{C} = \Box(e_2)$): The corresponding classification is: $\Delta; \Gamma \vdash \Box(e_2) \text{ at } \Uparrow \tau_2 \triangleright \Delta; \Gamma [\Uparrow \tau]$. The focus must synthesise type τ as per the focus mode, with the extra premises being $\tau \sqcup (\Box \Rightarrow \Box) \equiv \tau_1 \Rightarrow \tau_2$ and $\Delta; \Gamma \vdash e_2 \Downarrow \tau_1$.
- **Argument position** (e_2 is the focus, $\mathcal{C} = e_1(\Box)$): The corresponding classification is: $\Delta; \Gamma \vdash e_1(\Box) \text{ at } \Uparrow \tau_2 \triangleright \Delta; \Gamma [\Downarrow \tau_1]$. The focus analyses against τ_1 as per the focus mode, with the extra premises being: $\Delta; \Gamma \vdash e_1 \Uparrow \tau$ and $\tau \sqcup (\Box \Rightarrow \Box) \equiv \tau_1 \Rightarrow \tau_2$.

I give a few representative rules of context classification in Figure 16. The construction of these rules is systematic, one for each typing rule and focus position. The only exception being the identity contexts $\mathbf{s}\Box$ and $\mathbf{a}\Box$ (the base cases) saying that the focus mode must match the outer derivation mode.

$\boxed{\Delta; \Gamma \vdash \mathcal{C} \text{ at } d \triangleright \Delta'; \Gamma' [m]}$ Context $\mathcal{C}$ at outer derivation mode d classifies its focus under $\Delta'; \Gamma'$ in focus mode m

Base cases identity contexts (focus mode matches outer derivation mode):

$$\mathbf{s}\Box \frac{}{\Delta; \Gamma \vdash \Box \text{ at } \Uparrow \tau \triangleright \Delta; \Gamma [\Uparrow \tau]} \quad \mathbf{a}\Box \frac{}{\Delta; \Gamma \vdash \Box \text{ at } \Downarrow \tau \triangleright \Delta; \Gamma [\Downarrow \tau]}$$

Synthesis mode: $d = \Uparrow \tau$ the overall expression $\mathcal{C}[e]$ synthesises:

$$\mathbf{s}\circ_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \Uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup (\Box \Rightarrow \Box) \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash e_2 \Downarrow \tau_1}{\Delta; \Gamma \vdash \mathcal{C}(e_2) \text{ at } \Uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\ \mathbf{s}\circ_2 \frac{\Delta; \Gamma \vdash e_1 \Uparrow \tau \quad \tau \sqcup (\Box \Rightarrow \Box) \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash \mathcal{C} \text{ at } \Downarrow \tau_1 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e_1(\mathcal{C}) \text{ at } \Uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]}$$

Subsumption: $\Uparrow \tau' \rightsquigarrow \Downarrow \tau$ the unique bridge from synthesis to analysis derivation mode:

$$\mathbf{aSub} \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \Uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \tau \sim \tau'}{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \Downarrow \tau \triangleright \Delta'; \Gamma' [m]}$$

Figure 16: Selected context classification rules.

Correctness of context classification is formalised via a *totality* theorem: every well-typed expression can be decomposed into context and expression at some focus, with a context classification matching a (consistent) typing derivation of the focused term. Conversely, a *soundness* theorem states that a valid classification and expression satisfying the focus typing mode can be composed into a well-typed plugged expression.

Theorem 3.11 (⊙ Plug Decomposition – Totality). *For any context $\mathcal{C}$, expression e , and outer derivation mode d :*

- *If $\Delta; \Gamma \vdash \mathcal{C}[e] \uparrow \tau$, then there exist Δ' , Γ' , and m such that $\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]$ and e satisfies focus mode m under $\Delta'; \Gamma'$.*
- *If $\Delta; \Gamma \vdash \mathcal{C}[e] \downarrow \tau$, then there exist Δ' , Γ' , and m such that $\Delta; \Gamma \vdash \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]$ and e satisfies focus mode m under $\Delta'; \Gamma'$.*

Here “ e satisfies focus mode m under $\Delta'; \Gamma'$ ” means: if $m = \uparrow \tau'$ then $\Delta'; \Gamma' \vdash e \uparrow \tau'$; if $m = \downarrow \tau'$ then $\Delta'; \Gamma' \vdash e \downarrow \tau'$.

Theorem 3.12 (⊙ Plug Composition – Soundness). *For any context $\mathcal{C}$, expression e , outer derivation mode d , and focus mode m : if $\Delta; \Gamma \vdash \mathcal{C} \text{ at } d \triangleright \Delta'; \Gamma' [m]$ and e satisfies focus mode m under $\Delta'; \Gamma'$, then $\mathcal{C}[e]$ satisfies outer derivation mode d under $\Delta; \Gamma$.*

3.5 ⊙ Metatheory: Graduality & Unicity

Semantics/
Graduality

Finally, the static gradual guarantee (section 2.3), standard in gradual typing systems, is an essential monotonicity theorem used to ensure efficient computability of type slices.

Theorem 3.13 (⊙ Static Gradual Guarantee – Synthesis). *If $\Gamma' \sqsubseteq \Gamma$, $e' \sqsubseteq e$, and $\Delta; \Gamma \vdash e \uparrow \tau$, then there exists τ' such that $\Delta; \Gamma' \vdash e' \uparrow \tau'$ and $\tau' \sqsubseteq \tau$.*

Theorem 3.14 (⊙ Static Gradual Guarantee – Analysis). *If $\Gamma' \sqsubseteq \Gamma$, $e' \sqsubseteq e$, $\tau' \sqsubseteq \tau$, and $\Delta; \Gamma \vdash e \downarrow \tau$, then $\Delta; \Gamma' \vdash e' \downarrow \tau'$.*

The same monotonicity extends to context classifications, where $m' \sqsubseteq m$ lifts type precision through the focus mode constructors: $\uparrow \tau' \sqsubseteq \uparrow \tau$ iff $\tau' \sqsubseteq \tau$, and likewise for $\downarrow$. The focus modes itself (analysis or synthesis) is propagated unchanged through every classification rule.

Theorem 3.15 (⊙ Static Gradual Guarantee – Context, Synthesis Position). *If $\Gamma' \sqsubseteq \Gamma$, $\mathcal{C}' \sqsubseteq \mathcal{C}$, and $\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau_p \triangleright \Delta_f; \Gamma_f [m]$, then there exist τ'_p , Γ'_f , and m' with $\tau'_p \sqsubseteq \tau_p$, $\Gamma'_f \sqsubseteq \Gamma_f$, $m' \sqsubseteq m$, and $\Delta; \Gamma' \vdash \mathcal{C}' \text{ at } \uparrow \tau'_p \triangleright \Delta_f; \Gamma'_f [m']$.*

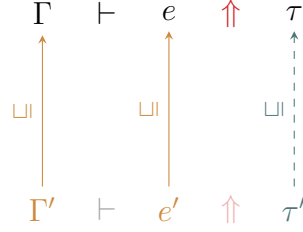


Figure 17: The static gradual guarantee – Synthesis (Theorem 3.13).

static-gradual-
ana-cls

Theorem 3.16 (○ Static Gradual Guarantee – Context, Analysis Position). *If $\Gamma' \sqsubseteq \Gamma$, $\mathcal{C}' \sqsubseteq \mathcal{C}$, $\tau'_p \sqsubseteq \tau_p$, and $\Delta; \Gamma \vdash \mathcal{C}$ at $\Downarrow_{\tau_p} \triangleright \Delta_f; \Gamma_f [m]$, then there exist Γ'_f and m' with $\Gamma'_f \sqsubseteq \Gamma_f$, $m' \sqsubseteq m$, and $\Delta; \Gamma' \vdash \mathcal{C}'$ at $\Downarrow_{\tau'_p} \triangleright \Delta_f; \Gamma'_f [m']$.*

Theorem 3.17 (○ Synthesis Unicity). *If $\Delta; \Gamma \vdash e \Uparrow \tau_1$ and $\Delta; \Gamma \vdash e \Uparrow \tau_2$, then $\tau_1 = \tau_2$.*

Corollary 3.18 (○ Synthesis Precision). *If $\Gamma' \sqsubseteq \Gamma$, $e' \sqsubseteq e$, $\Delta; \Gamma \vdash e \Uparrow \tau$, and $\Delta; \Gamma' \vdash e' \Uparrow \tau'$, then $\tau' \sqsubseteq \tau$.*

Mechanisation provenance. The Agda mechanisation of graduality on expression typing (theorems 3.13 and 3.14) is loosely based on [hazeltrove/hazelnut-polymorphism-agda](#), but had to be re-proven from scratch to integrate with the Type Slicing core calculus.⁶ The graduality theorems for context classification (theorems 3.15 and 3.16) are new to this work, since context classification itself is novel.

⁶ Which is sufficiently similar to Hazel for this provenance to be useful.

4 Synthesis Slices

Legend: $\bigcirc$ fully mechanised, $\bigodot$ mechanised with postulates, $\triangle$ not mechanised, $\square$ important theorem, $\square$ counterexample.

The type information of a sub-term in a bidirectional type system is derived either from *within* the term by synthesis, or from the surrounding context. This chapter defines *synthesis slices* which concern how to explain an expression's *synthesised* type, that is, the type information *internal* to the expressions.

4.1 $\bigcirc$ Synthesis Slices

Slicing/Synthesis/
Synthesis

Consider a ‘program’ $P = (\Gamma, e)$, that is, typing assumptions and an expression. When the program synthesises a type (in some type variable context Δ)⁷, all of its type information is contained within the synthesis derivation:

$$D : \Delta; \Gamma \vdash e \Uparrow \tau$$

Synthesis slices work in the following manner: Given the synthesis derivation D , we may *query* the type τ by taking a slice v in $[\tau]$. A resulting synthesis slice is a ‘program slice’ $\rho = (\gamma, \varsigma)$ in $[\Gamma, e]$ which synthesises a type at least as precise as the query v ; this corresponds to a highlighted version of the original program.

Intuitively, v specifies the part of τ we wish to explain; for example, querying $v = \text{Int} \Rightarrow \square$ for $\tau = \text{Int} \Rightarrow \text{Bool}$ asks “which parts of the program account for the domain being *Int*?”. Importantly, the resulting program slice might not include *all* the program regions relevant to the query type, but will necessarily provide a consistent subset which suffice to totally *explain* (independently synthesise) the query.

Definition 4.1 ($\bigcirc$ Synthesis slice). A *synthesis slice* of $D : \Delta; \Gamma \vdash e \Uparrow \tau$ on $v \in [\tau]$, written $\text{SynSlice } D \blacktriangleleft v$, is a pair $(\gamma, \varsigma) \in [\Gamma, e]$ such that $\Delta; \gamma \vdash \varsigma \Uparrow \phi$ for some $\phi \sqsubseteq v$ (with $\phi \in [\tau]$ by graduality, theorem 3.13).

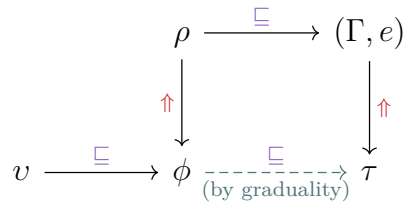


Figure 18: Diagram of a synthesis slice; $v \sqsubseteq \phi$ is the validity witness.

Many such synthesis slices exist. In particular, the original program (Γ, e) is necessarily a synthesis slice; and for $\text{SynSlice } D \blacktriangleleft \square$, every slice of (Γ, e) is a valid synthesis slice.

Exactness. Note that synthesis slices do not strictly demand $v = \phi$. I terms such slices that do as *exact* synthesis slices. Exact slices are an intuitive way to define synthesis slices, but we shall see that exactness is *too strong* a condition.

⁷ Type variable contexts are not sliced as the only information such slices would encode is whether a type variable was used, which can be extracted purely syntactically from the resulting slice. Typing assumptions are different because we can *partially* require an assumption.

Definition 4.2 ($\bigcirc$ Exact synthesis slice). *ExactSynSlice* $D \blacktriangleleft v$ is a synthesis slice $(\gamma, \varsigma) : \text{SynSlice } D \blacktriangleleft v$ whose synthesised type ϕ satisfies $\phi \sqsubseteq v$.

However, exact synthesis slices are too strong a constraint to use in general. They need not exist:

Counterexample 4.3 ($\bigcirc$ Non-Existence of all Exact Synthesis Slices). *Consider $x : * \Rightarrow * \vdash (x, x) \uparrow (* \Rightarrow *) \times (* \Rightarrow *)$, and query $v = (* \Rightarrow \square) \times (\square \Rightarrow *)$. Any strictly smaller slice of either $x : * \Rightarrow *$ or (x, x) synthesises a type strictly less precise than v . Yet the original program synthesises $(* \Rightarrow *) \times (* \Rightarrow *)$, strictly more precise than v .*

4.2 $\bigcirc$ Minimality

Synthesis slices give us *some* explanation of the query, but many are clearly useless, such as the original program itself, which corresponds to fully highlighting the program. We want to find minimal slices: those that further slicing any part of the program makes the slice invalid (synthesising a type insufficient to cover the query).

Definition 4.4 ($\bigcirc$ Minimality). For an element a of a poset, a is minimal, $\text{IsMinimal}(a)$, when all a' with $a' \sqsubseteq a$ are in fact equal to a , have $a' = a$.

Definition 4.5 ($\bigcirc$ Minimal Synthesis Slices). A *MinSynSlice* $D \blacktriangleleft v$ is a *SynSlice* $D \blacktriangleleft v$ minimal in *SynSlice* $D \blacktriangleleft v$ under program-slice precision on $[\Gamma, e]$.

4.3 $\triangle$ Existence of Minimal Slices

Crucially, any synthesis slice has a minimal slice below it (or is minimal):

Theorem 4.6 ($\triangle$ Existence of minimal slices). *For every $s : \text{SynSlice } D \blacktriangleleft v$, there exists $m : \text{MinSynSlice } D \blacktriangleleft v$ with $m \sqsubseteq s$.*

Proof. The slice lattice $[\Gamma, e]$ is finite. Hence, $\{s' : \text{SynSlice } D \blacktriangleleft v \mid s' \sqsubseteq s\}$ is finite, which can be decided simply by enumerating all $s' : \text{SynSlice } D \blacktriangleleft v$ and deciding $s' \sqsubseteq s$. If this set is empty, then s is minimal. $\square$

Consequently, any derivation D has a minimal synthesis slice, as the original program is a trivial inhabitant of *SynSlice* $D \blacktriangleleft v$ for all v .

The proof above is not a practical algorithm. In reality, for any term, you can efficiently compute the list of maximal slices that are strictly less precise than some given term (by omitting exactly *one* leaf from the AST). Then pick the first maximal strict slice that satisfies the query and recurse; if none of the slices satisfy the query, then you have found a minimal slice. This is $\mathcal{O}(n \times T)$ where n is the program size and T the type checking complexity; type checking is linear in the program size for this calculus, so finding a minimal slice is $\mathcal{O}(n^2)$.

See Figure 19 for an example lattice to explore, marking all minima (of which there may be multiple, Counterexample 4.7).

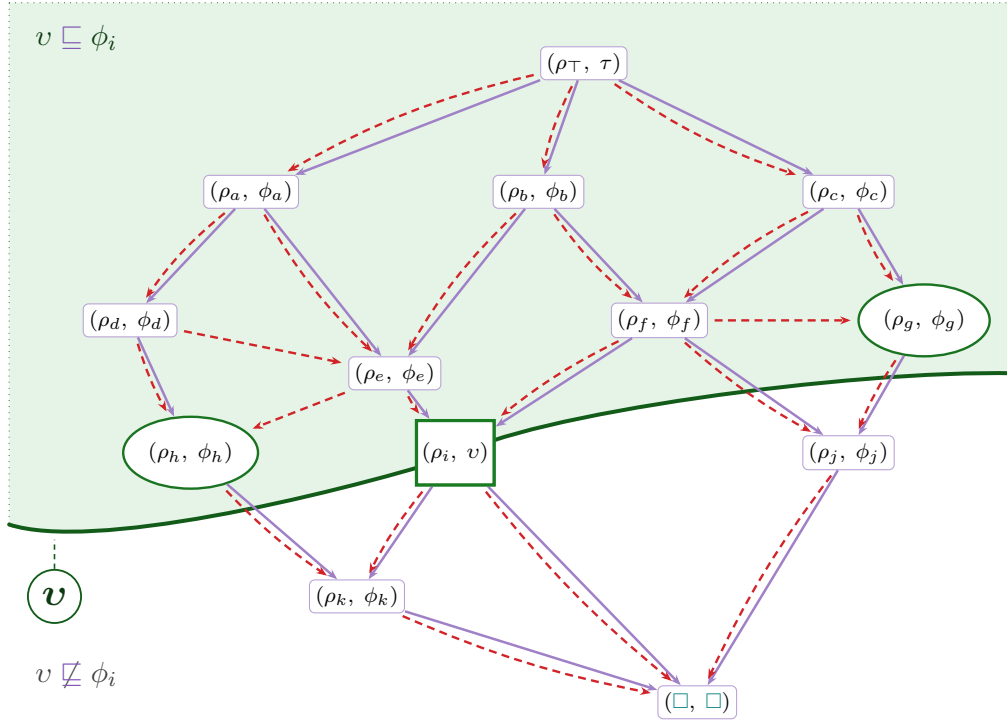


Figure 19: An example Hasse diagram of program slices in $[\rho_\tau]$ overlaid with type precision relations of their synthesised types (mostly induced by graduality). Classified into a shaded region of *valid synthesis slices*, synthesising $\phi_i \sqsubseteq v$.

Purple: program precision $\sqsubseteq$. Red: type precision $\sqsubseteq$. Ellipses: minimal synthesis slices. Square: an exact minimal slice

Counterexample 4.7 ($\triangle$ Non-Uniqueness of Minimal Slices). *The program*
if true then * else * *has two incomparable minimal slices for* $v = *$:
if $\square$ then * else $\square$ *and* **if $\square$ then $\square$ else ***.

Refining a query v to a smaller $v' \sqsubseteq v$ descends further in the lattice, finding minima which are less precise, but consistent with, the previous slice for v .

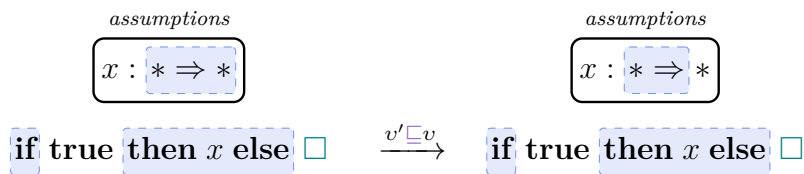
Theorem 4.8 ($\triangle$ Monotonicity of minimal slices). *If $v_1 \sqsubseteq v_2$ in $[\tau]$ and $m_2 : \text{MinSynSlice } D \blacktriangleleft v_2$, then there exists $m_1 : \text{MinSynSlice } D \blacktriangleleft v_1$ with $m_1 \sqsubseteq m_2$.*

4.4 $\bigcirc$ Sub-slices

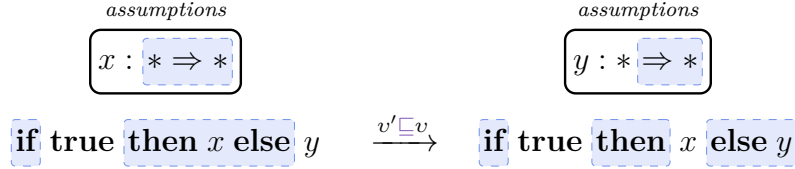
Slicing/Synthesis/
FixedAssmsCalc

In practice, a user may not arrive at a query atomically. One might look at a synthesised type, then refine the query to explore one component of it, then into a sub-component. For this use case we get successive queries forming a descending chain $v \sqsupseteq v' \sqsupseteq v'' \sqsupseteq \dots$. Due to monotonicity, this chain can be associated with a chain of program slices $\rho \sqsupseteq \rho' \sqsupseteq \dots$.

Example. Refining $v = * \Rightarrow * \Rightarrow \square$ on $e = \text{if true then } x \text{ else } y$:



Branch hopping. This descending chain of program slices is useful because it intuitively aligns with the users thinking of *refining* an existing slice, and retrieving a refined program consistent with the last query. Picking random minimal slices will not necessarily form such a chain, giving rise to ‘branch hopping’ where highlighted regions switch between branches when refining the query, which is undesirable from a UX perspective. For example, refining $v = * \Rightarrow *$ to $v' = \square \Rightarrow *$:



Slicing/Synthesis/
Decompositions

4.5 ○ Decompositions

Instead of calculating slices by brute force (section 4.3), it would be ideal if we could calculate minimal slices of a term from minimal subslices of its subterms. This section proves that any minimal slice on a compound can indeed be created by composing minimal slices on its subterm, suggesting that such approaches are indeed possible; these are further explored in the optimisations of section 7.

Firstly, each compound typing rule must come with a *slice combinator*: an operation that builds a synthesis slice from slices of the typing rule’s premises, joining their shared assumptions. Then, we must be able to decompose a compound minimal slice into minimal slices combined using the corresponding combinator.

I give the two of the cases: the product rule and the let binding rule. The remaining cases are similar.

4.5.1 ○ Products

Given slices $(\gamma_1, \varsigma_1) : \text{SynSlice } D_1 \blacktriangleleft v_1$ and $(\gamma_2, \varsigma_2) : \text{SynSlice } D_2 \blacktriangleleft v_2$ for the two components of a pair $D = \uparrow\text{Pair } D_1 D_2$ (with synthesised types ϕ_1, ϕ_2), we form their product slice $(\gamma_1, \varsigma_1) \times (\gamma_2, \varsigma_2) : \text{SynSlice } D \blacktriangleleft v_1 \times v_2$ by joining the two assumption slices and pairing the term slices:

$$(\gamma_1, \varsigma_1) \times (\gamma_2, \varsigma_2) \triangleq (\gamma_1 \sqcup \gamma_2, (\varsigma_1, \varsigma_2))$$

The pair synthesises (possibly more precise)⁸ types $\phi'_1 \times \phi'_2 \sqsubseteq \phi_1 \times \phi_2 \sqsubseteq v_1 \times v_2$. For this combinator, we get the following decomposition theorem:

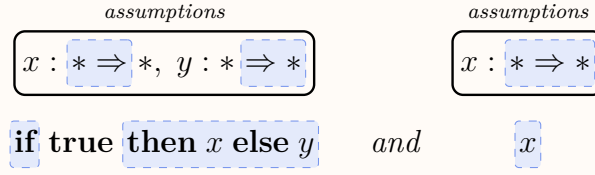
min-prod-
decomposability

Theorem 4.8.1 (○ Case: Products). *Let $m : \text{MinSynSlice } \uparrow\text{Pair } D_1 D_2 \blacktriangleleft v_1 \times v_2$. Then there exist minimal $m_1 : \text{MinSynSlice } D_1 \blacktriangleleft v_1$ and $m_2 : \text{MinSynSlice } D_2 \blacktriangleleft v_2$ with $m = m_1 \times m_2$.*

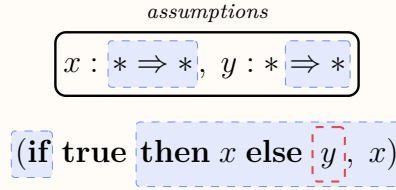
○ **Converse Failure** However, the converse is not true:

⁸ Due to $\gamma_1 \sqcup \gamma_2$ being larger than γ_1 and γ_2

Counterexample 4.9 (○ Paired minimal slices are not necessarily minimal). Consider two minimal slices, both on query $* \Rightarrow *$:



Note that left slice is minimal due to the assumption slice, omitting either x or y would no longer synthesise $* \Rightarrow *$. However, the pair combinator, which pairs the expression slices and joins the assumption slices, produces:



Which is not minimal, the y outlined in red can be removed while still synthesising $(* \Rightarrow *) \times (* \Rightarrow *)$ (the natural product of the queries of the components).

Hence, we cannot simply recurse through the typing derivation to calculate a slice on $v_1 \times v_2$ from sub-slices of the components on v_1 and v_2 respectively. Section 7 solves this issue.

4.5.2 ○ Let Bindings

A more subtle case is of let-bindings, which has difference assumptions for its two premises:

$$\uparrow\text{Let} \frac{\Delta; \Gamma \vdash e_1 \uparrow \tau_1 \quad \Delta; \Gamma, x : \tau_1 \vdash e_2 \uparrow \tau_2}{\Delta; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \uparrow \tau_2}$$

A slice of a let-expression can be constructed from a slice (γ_1, ς_1) of the definition (synthesising ϕ_1) and a slice (γ_2, ς_2) of the body, constrained with an assumption consistency condition: the body's used assumptions for the bound variable x must be *at most as precise as* the type synthesised by the slice of the definition:

$$\gamma_2(x) \sqsubseteq \phi_1$$

Then, upon joining the assumptions for the two slices, we must remove the assumption on x from the body (notated $\gamma_{\setminus x}$), as this is now provided by the definition:

$$(\gamma_1, \varsigma_1) \text{ def } (\gamma_2, \varsigma_2) \triangleq (\gamma_1 \sqcup (\gamma_2)_{\setminus x}, \text{let } x = \varsigma_1 \text{ in } \varsigma_2)$$

Theorem 4.8.2 (Case: Bindings). Let $m : \text{MinSynSlice } \uparrow\text{Let } D_1 D_2 \blacktriangleleft v$. Then there exists $\phi_1 \in [\tau_1]$ and minimal slices $m_1 : \text{MinSynSlice } D_1 \blacktriangleleft \phi_1$ (synthesising ϕ'_1) and $m_2 = (\gamma_2, \varsigma_2) : \text{MinSynSlice } D_2 \blacktriangleleft v$ satisfying $\gamma_2(x) \sqsubseteq \phi'_1$, with $m = m_1 \text{ def } m_2$.

4.6 $\bigcirc$ Contribution Slices

A natural way represent a ‘least’ slice that retains *all* information relevant to a query type query, rather than just enough to consistently explain the type, would be to *join* all minimal slices for the slices. These joins are necessarily unique and are ‘least’ in the sense of being a least upper bound of actual minimal slices.

For this to make sense, closure of synthesis slices under joins is the crucial, joining two synthesis slices explaining v should still explain v fully. This is a second instance where exact synthesis slices are too strong, we do *not* get that joining two exact slices synthesises exactly the joined query:

 $\neg \sqcup_{\text{syn-closed}}$

Counterexample 4.10 ($\bigcirc$ Joins do not preserve exactness). Consider $D : (x : *) \vdash x \uparrow *$. For the query $v = \square$, consider two exact (but not minimal) slices: $x : \square \vdash x \uparrow \square$ and $x : * \vdash \square \uparrow \square$. However, their join synthesises $* : x : * \vdash x \uparrow *$, despite the join of the queries being $\square$. This synthesis is not exact.

Even if we restrict this the exact slices to be minimal:

 $\neg \sqcup_{\text{syn-preserves-join}}$

Counterexample 4.11 ($\bigcirc$ Joins do not preserve exactness, even under minimality). Consider $D = x : * \Rightarrow * \vdash (x, x) \uparrow (* \Rightarrow *) \times (* \Rightarrow *)$. Then for queries $v_1 = \square \times (\square \Rightarrow *)$ and $v_2 = (* \Rightarrow \square) \times \square$, we get minimal slices $(x : \square \Rightarrow *, (\square, x))$ and $(x : * \Rightarrow \square, (x, \square))$ respectively. Joining these minimal slices gives the original derivation D , which synthesises $(* \Rightarrow *) \times (* \Rightarrow *) \sqsubseteq (* \Rightarrow \square) \times (\square \Rightarrow *) = v_1 \times v_2$.

However, the weaker validity condition for regular synthesis slices $v \sqsubseteq \phi$, does allow for join closure.

 $\sqcup_{\text{syn}}$

Theorem 4.12 ($\bigcirc$ Closure of Synthesis Slices under Join). For $(\gamma_1, \varsigma_1) : \text{SynSlice } D \blacktriangleleft v_1$ and $(\gamma_2, \varsigma_2) : \text{SynSlice } D \blacktriangleleft v_2$, then $(\gamma_1, \varsigma_1) \sqcup (\gamma_2, \varsigma_2) : \text{SynSlice } D \blacktriangleleft v_1 \sqcup v_2$, with the joined slice taken pointwise: $(\gamma_1 \sqcup \gamma_2, \varsigma_1 \sqcup \varsigma_2)$.

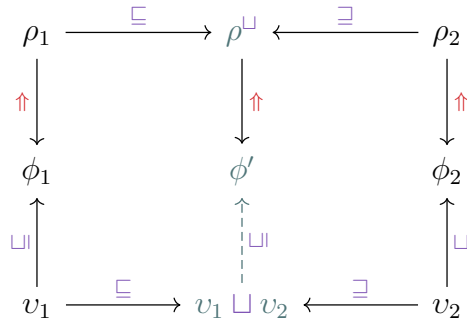


Figure 20: Theorem 4.12.

5 Analysis Slices

Legend: $\bigcirc$ fully mechanised, $\bigtriangleup$ mechanised with postulates, $\triangleleft$ not mechanised, $\square$ important theorem, $\square$ counterexample.

Analysis slices explain why a sub-terms is *analysed* against an expected type. The principal difference from synthesis slices is the object being sliced: where a synthesis slice sliced a derivation $\Delta; \Gamma \vdash e \uparrow \tau$, an analysis slice slices a *context classification* $\Delta; \Gamma \vdash \mathcal{C} \text{ at } d \triangleright \Delta'; \Gamma' [m]$ around the selected term. The expression slice ς is replaced by a context slice $\kappa \in [\mathcal{C}]$; everything else (minimality, existence, monotonicity) proceeds analogously to Section 4.

5.1 $\bigcirc$ Analysis Slices

Slicing/Analysis/
Analysis

Analysis slices concern the classifications with focus mode $m = \Downarrow \tau$. A query $v \in [\tau]$ specifies the part of τ that we wish to explain.

Unlike synthesis slices, there are two possible outer derivations modes d . Analysis slices are for sub-terms within a synthesising expression, but the alternative concept, *inner* analysis slices, are for when the whole term is being analysed.

Definition 5.1 ($\bigcirc$ Analysis slice). An *analysis slice* of $\mathcal{C}ls : \Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau_p \triangleright \Delta'; \Gamma' [\Downarrow \tau]$ on $v \in [\tau]$, written $AnaSlice \mathcal{C}ls \blacktriangleleft v$, is a pair $(\kappa, \gamma) \in [\mathcal{C}, \Gamma]$ such that $\Delta; \gamma \vdash \kappa \text{ at } \uparrow \psi \triangleright \Delta''; \Gamma'' [\Downarrow v]$ for some outer-type witness $\psi \sqsubseteq v$ (with $\psi \in [\tau_p]$ by graduality, theorem 3.15).

Definition 5.2 ($\bigcirc$ Inner analysis slice). An *inner analysis slice* of $\mathcal{C}ls : \Delta; \Gamma \vdash \mathcal{C} \text{ at } \Downarrow \tau_p \triangleright \Delta'; \Gamma' [\Downarrow \tau]$ on $v \in [\tau]$, written $InnerAnaSlice \mathcal{C}ls \blacktriangleleft v$, is a pair $(\kappa, \gamma) \in [\mathcal{C}, \Gamma]$ such that, for some chosen outer-type $v_{outer} \in [\tau_p]$ then $\Delta; \gamma \vdash \kappa \text{ at } \Downarrow v_{outer} \triangleright \Delta''; \Gamma'' [\Downarrow v]$.

5.2 $\bigcirc$ Minimality

Slicing/Analysis/
Analysis

$MinAnaSlice \mathcal{C}ls \blacktriangleleft v$ defined analogously to Definition 4.4, minimising over the context slice (κ, γ) . Similarly for $MinInnerAnaSlice \mathcal{C}ls \blacktriangleleft v$, except v_{outer} is also minimised, i.e. we additionally find a minimal outer type to the whole expressions check against, that still checks the focus at our query v .

As before, minimal analysis slices need not be unique, but this non-determinism is wholly inherited from synthesis.

5.3 $\bigtriangleup$ Existence of Minimal Slices

Slicing/Analysis/
Analysis

Theorem 5.3 ($\bigtriangleup$ Existence of minimal analysis slices). *For every $s : AnaSlice \mathcal{C}ls \blacktriangleleft v$, there exists $m : MinAnaSlice \mathcal{C}ls \blacktriangleleft v$ with $m \sqsubseteq s$; and analogously for $InnerAnaSlice$.*

Proof. The slice lattice $[\mathcal{C}, \Gamma]$ is finite, so $\{(\kappa', \gamma') \sqsubseteq (\kappa, \gamma) \mid (\kappa', \gamma') \text{ is a slice}\}$ is finite. Enumerate it and descend; the chain terminates at a minimal element. $\square$

Theorem 5.4 ($\bigtriangleup$ Monotonicity of minimal analysis slices). *If $v_1 \sqsubseteq v_2$ in $[\tau]$ and $m_2 : MinAnaSlice \mathcal{C}ls \blacktriangleleft v_2$, then there exists $m_1 : MinAnaSlice \mathcal{C}ls \blacktriangleleft v_1$ with $m_1 \sqsubseteq m_2$; analogously for $InnerAnaSlice$.*

6 Error Marking & Type Error Debugging

Legend: $\bigcirc$ fully mechanised, $\bigodot$ mechanised with postulates, $\triangle$ not mechanised, $\square$ important theorem, $\square$ counterexample.

This chapter associates the tools of this formalism with the error squiggles of fig. 2 (section 1.1). It demonstrates that we can not only use this calculus to “explain types” in well-typed programs, but also effectively “explain type errors” in ill-typed programs.

This builds upon the theory of type error marking for bidirectional systems with holes of Zhao et al. [37], which I briefly recap and extend to the type slicing core calculus (section 3) in the next subsection.

6.1 $\bigcirc$ Error Marking

A marking algorithm is a pair of bidirectional judgements,

$$\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \text{and} \quad \Delta; \Gamma \vdash e \multimap \check{e} \downarrow \tau,$$

reading “under $\Delta; \Gamma$, the source expression e *marks to* the marked expression $\check{e}$, synthesising (resp. being analysed against) type τ ”. The marked output $\check{e}$ has the same structure as e but wraps each point of local type failure in a *mark*. The marks, representing highlighting an entire erroneous term e , are tabulated in table 1 alongside the corresponding typing rule failures.

The core calculus studied gives rise to four classes of errors: *type inconsistencies* when subsumption cannot be applied, *shape errors* when rules expect a sub-term to join (match) a specific type shape, *mode limitations* where syntax which cannot synthesise its type is expected to provide type information. And variable *scoping errors*, which do not pertain to types, so cannot be debugged by type slicing.

Mark	Class	Failed Rule(s)	Error
$\langle e \rangle_{\tau' \not\sim \tau}$	type inconsistency	Subsumption	e synthesises τ' inconsistent with analysis type τ
$\langle e \rangle_{\blacktriangleright \Rightarrow}$	shape	Function Application	e is in a function position, but is not a function
$\langle e \rangle_{\blacktriangleright +}$	shape	Case Expression	e is a case scrutinee, but is not a sum type
$\langle e \rangle_{\blacktriangleright \times}$	shape	Product Projection	e is projected as a product, but is not a product
$\langle e \rangle_{\blacktriangleright \forall}$	shape	Typfun Application	e is in a typfun position, but is not a typfun
$\langle \lambda x. e \rangle_{\sim \Rightarrow}$	mode limitation	Lambda Synthesis	Unannotated lambda in synthesis position
$\langle \lambda x. e \rangle_{\sim \Rightarrow}$	mode limitation	Lambda Analysis	Unannotated lambda analysed against a non-function type
$\langle k \rangle_{\uparrow}$	scope	Variables	Variable k has no binding in scope

Table 1: Error Mark Forms Classified

Importantly, marked terms can be type-checked by treating the marked terms as ‘non-empty’ holes, synthesising $\square$ (and recursively marking and type-checking the inner term).

The validity of this method is formalised by two theorems: *totality* ensures marks account for every possible typing error, and *erasure* ensures adding marks doesn't change the structure of the terms.

Theorem 6.1 (Totality of Marking). *For every expression e and assumptions $\Delta; \Gamma$. There exists marked expression $\check{e}$ and type τ such that $\Delta; \Gamma \vdash e \Downarrow \check{e} \Uparrow \tau$. And, for every type τ' , there exists $\check{e}'$ such that $\Delta; \Gamma \vdash e \Downarrow \check{e}' \Downarrow \tau$.*

Theorem 6.2 (Erasure of Marking). *If $\Delta; \Gamma \vdash e \Downarrow \check{e} \Uparrow \tau$ then $\text{erase}(\check{e}) \equiv e$, and likewise for analysis.*

Mechanisation provenance. The Agda development is loosely based on the mechanisation accompanying Zhao et al. [37] – [hazelgrove/error-localization-agda](https://hazelgrove.github.io/error-localization-agda/) – but the proofs are not direct ports. They were re-derived from scratch against the core calculus of section 3, following the recipe proposed in the paper.

6.2 Marked Context Classification

Semantics/Marking/
CtxMarking

Context classification (section 3.4) gives a reading of where a one-hole context C sits in a derivation. This can be extended to marked context classification $\Delta; \Gamma \vdash C \Downarrow \check{C} \text{ at } p \triangleright \Delta'; \Gamma' [m]$ threads a marked context $\check{C}$ alongside C (with context marks defined analogously to expressions), with one error rules per mark form in table 1.

For example, the type inconsistency mark can be inserted into context classification with the following rule:

$$\text{maSub}\Uparrow \frac{\Delta; \Gamma \vdash C \Downarrow \check{C} \text{ at } \Uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \neg(\tau \sim \tau')}{\Delta; \Gamma \vdash C \Downarrow (\check{C})_{\tau' \not\sim \tau} \text{ at } \Downarrow \tau \triangleright \Delta'; \Gamma' [m]}$$

The marked context wraps the inner marked context $\check{C}$ in an inconsistency mark when the analysed and synthesised types of the *full plugged expression* differ: $\tau' \not\sim \tau$. The full rule set is collected in section A.

A Worked Decomposition Take the expression:

$$\text{let } p : * \times (* \Rightarrow *) = (*, \text{true}) \text{ in } p.$$

The pair's right component synthesises `Bool` but is analysed against $* \Rightarrow *$ (from the annotation), so marking should produce:

$$\text{let } p : * \times (* \Rightarrow *) = (*, (\text{true})_{\text{Bool} \not\sim * \Rightarrow *}) \text{ in } p.$$

Placing the focus on the marked term `true` gives:

$$C = \text{let } p : * \times (* \Rightarrow *) = (*, \bigcirc) \text{ in } p, \quad e = \text{true}.$$

The classification builds $\check{C}$ from the inside out, first marking the focus with the type inconsistency:

Step (rule)	$\check{C}$ so far
mso	$\bigcirc$
maSub $\Uparrow$	$(\bigcirc)_{\text{Bool} \not\sim * \Rightarrow *}$
ma& ₂	$(*, (\bigcirc)_{\text{Bool} \not\sim * \Rightarrow *})$
malet...	$\text{let } p : * \times (* \Rightarrow *) = (*, (\bigcirc)_{\text{Bool} \not\sim * \Rightarrow *}) \text{ in } p$

Plugging gives $\check{C}[\mathbf{true}]$:

$$\mathbf{let } p : * \times (* \Rightarrow *) = (*, (\mathbf{true})_{\text{Bool} \not\sim * \Rightarrow *}) \mathbf{in } p.$$

Marked context classification is validated with a pair of theorems mirroring the unmarked *plug* totality and soundness theorems (theorems 3.11 and 3.12).

mplug-
decompose

Theorem 6.3 (○ Marked Plug Decomposition – Totality). *For any context $\mathcal{C}$, expression e , and outer derivation mode d :*

- If $\Delta; \Gamma \vdash \mathcal{C}[e] \Downarrow \check{e}_* \Uparrow \tau$, then there exist $\Delta', \Gamma', \check{C}, \check{e}$, and m such that $\Delta; \Gamma \vdash \mathcal{C} \Downarrow \check{C}$ at $\Uparrow \tau \triangleright \Delta'; \Gamma' [m]$, e marks to $\check{e}$ at focus mode m , and $\check{e}_* = \check{C}[\check{e}]$.
- If $\Delta; \Gamma \vdash \mathcal{C}[e] \Downarrow \check{e}_* \Downarrow \tau$, then there exist $\Delta', \Gamma', \check{C}, \check{e}$, and m such that $\Delta; \Gamma \vdash \mathcal{C} \Downarrow \check{C}$ at $\Downarrow \tau \triangleright \Delta'; \Gamma' [m]$, e marks to $\check{e}$ at focus mode m , and $\check{e}_* = \check{C}[\check{e}]$.

Here “ e marks to $\check{e}$ at focus mode m ” means: if $m = \Uparrow \tau'$ then $\Delta'; \Gamma' \vdash e \Downarrow \check{e} \Uparrow \tau'$; if $m = \Downarrow \tau'$ then $\Delta'; \Gamma' \vdash e \Downarrow \check{e} \Downarrow \tau'$.

mplug-
compose

Theorem 6.4 (○ Marked Plug Composition – Soundness). *For any context $\mathcal{C}$, marked context $\check{C}$, outer derivation mode d , and focus mode m : if $\Delta; \Gamma \vdash \mathcal{C} \Downarrow \check{C}$ at $d \triangleright \Delta'; \Gamma' [m]$ and e marks to $\check{e}$ at focus mode m , then $\mathcal{C}[e]$ marks to $\check{C}[\check{e}]$ at outer derivation mode d .*

6.3 The Debugging Recipe by Example

We may now relate the tools developed over the last 4 sections to explain type errors:

1. Place the focus (cursor) at some marked term. This splits the marked program into a marked context $\check{C}$ and marked focus $\check{e}$, with $\check{C}[\check{e}]$ recovering the program; a marked context classification at the focus is then derivable by totality (theorem 6.3).
2. Synthesis slice the internal type information, the term at the *focus*, $\check{e}$, if the focus mode m is in synthesis ($\Uparrow \tau$).
3. Analysis slice the external type information, the *context classification* itself, if the focus mode m is in analysis ($\Downarrow \tau$).
4. Pick queries to explore the error. For type-inconsistency marks both the synthesis and analysis slices exist (context classifications for both steps 2 and 3 exist), and you can query *only* the inconsistent parts. For shape errors, query just the outermost shape of the synthesised type; mode-limitation errors are analogous on the analysis side.

I give some worked examples, with the following notation:

- The marked term itself is wrapped in a red dashed box.
- The synthesis slice of the focus is highlighted in blue.

- The analysis slice of the surrounding context is highlighted in green. With purely structural parts which don't contribute to the type (i.e. bindings) in a lighter shade.⁹

6.3.1 Type inconsistency: $(\cdot)_{\tau'} \not\sim_{\tau}$

Consider the program

let $p : * \Rightarrow * = (\text{true}, \text{false})$ **in** p .

The RHS $(\text{true}, \text{false})$ synthesises $\text{Bool} \times \text{Bool}$ but is analysed against $* \Rightarrow *$ – inconsistent head constructors. Marking inserts $(\cdot)_{\text{Bool} \times \text{Bool} \not\sim * \Rightarrow *}$ around the RHS:

let $p : * \Rightarrow * = ((\text{true}, \text{false}))_{\text{Bool} \times \text{Bool} \not\sim * \Rightarrow *} \text{in } p$.

Decomposition. The (context, focus) pair the marking factors into is

$C = \text{let } p : * \Rightarrow * = \bigcirc \text{ in } p, \quad e = (\text{true}, \text{false}),$

with the focus at $\Downarrow(* \Rightarrow *)$. The two debugging queries run on the two sides of this split.

Synthesis side (blue, on the focus e). *MinSynSlice* at the minimal type-slice $v \sqsubseteq \text{Bool} \times \text{Bool}$ that still witnesses $v \not\sim (* \Rightarrow *)$:

$e \rightsquigarrow (\text{true}, \text{false}) \quad \text{with} \quad v = \square \times \square.$

Analysis side (green, on the context C). *MinAnaSlice* at the minimal $\psi \sqsubseteq * \Rightarrow *$ that still witnesses $(* \times *) \not\sim \psi$:

$C \rightsquigarrow \text{let } p : * \Rightarrow * = \bigcirc \text{ in } p \quad \text{with} \quad \psi = \square \Rightarrow \square.$

Full view.

synthesis query $v = \text{Bool} \times \text{Bool} \quad (\equiv \square \times \square)$
analysis query $\psi = * \Rightarrow * \quad (\equiv \square \Rightarrow \square)$

let $p : * \Rightarrow * = ((\text{true}, \text{false}))_{\text{Bool} \times \text{Bool} \not\sim * \Rightarrow *} \text{in } p$.

The slices pick out exactly what each side contributes to the failure.

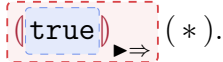
The recipe approach does not arbitrarily blame either the synthesising term or the analysing context for the error. A user who does want to trust the surrounding context (typically, annotations) as is often done in conventional type error highlighting can simply restrict the queries to include only the synthesis-side.

⁹ This idea of 'purely' structural parts is not formalised here, but is easy to implement in practice.

6.3.2 Shape mismatches: $(\cdot)_{\blacktriangleright\Rightarrow}, (\cdot)_{\blacktriangleright+}, (\cdot)_{\blacktriangleright\times}, (\cdot)_{\blacktriangleright\forall}$

The shape-mismatch marks $(\cdot)_{\blacktriangleright\Rightarrow}, (\cdot)_{\blacktriangleright+}, (\cdot)_{\blacktriangleright\times}, (\cdot)_{\blacktriangleright\forall}$ all share the same simple recipe: the syntactic position of the focus requires a type kind, but the focus's synthesised type does not match this.

A representative example is $e = \text{true}(*):$ the function position synthesises `Bool`, which is a function, i.e. does not join with $\square \Rightarrow \square$:

synthesis query $v = \text{Bool}$
 analysis query $\psi = -$


$(\cdot)_{\blacktriangleright+} (\text{case } * \text{ of } \dots), (\cdot)_{\blacktriangleright\times} (\pi_1 \text{ true}),$ and $(\cdot)_{\blacktriangleright\forall}$ are analogous.

6.3.3 Unannotated Lambda against Non-Arrow: $(\cdot)_{\sim\Rightarrow}$

Unannotated lambdas $\lambda y.e$ have no synthesis rule, type must be by the surrounding context. Consequentially it has no synthesis type, so type inconsistencies are not marked via subsumption, but instead by a mode limitation mark. In this situation all the relevant type information is derived from the surrounding context, so we use analysis slices.

Consider the program:

let $b : \text{Bool} = \lambda y.y$ **in** $b.$

The RHS $\lambda y.y$ is a bare lambda analysed against `Bool`. Take the *MinAnaSlice* on the enclosing context witnesses the context enforcing a non-function type (`Bool`):

synthesis query $v = -$
 analysis query $\psi = \text{Bool}$


6.3.4 Unbound variable: $\langle k \rangle \uparrow$

The free-variable mark records a *scope* failure: there are no types involved.

7 Optimising Type Slice Calculation

Legend: $\bigcirc$ fully mechanised, $\bigotimes$ mechanised with postulates, $\triangle$ not mechanised, $\square$ important theorem, $\square$ counterexample.

The theory of Sections 4–5 establishes *which* slices exist, and provide a brute-force algorithm. This section explores these structures further to improve the brute-force algorithm.

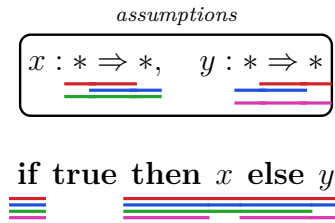
7.1 $\bigcirc$ Products of Synthesis Slices

Slicing/Synthesis/
FixedAssmsCalc

The product counterexample of Section 4.5.1 is worth re-examining, which stated that not all pairs of minimal slices can be paired into a minimal product slice. Consider the following if statement:

$$\Gamma = (x : * \Rightarrow *, y : * \Rightarrow *), \quad e = \text{if true then } x \text{ else } y, \quad \tau = * \Rightarrow *,$$

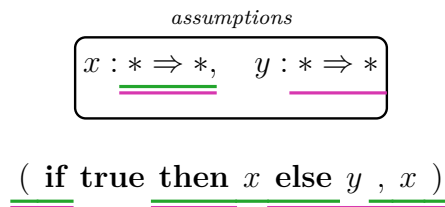
and ask for slices of $D : \Delta; \Gamma \vdash e \uparrow \tau$ at the full query $v = * \Rightarrow *$. There are *four* incomparable minimal full slices, displayed together as underlines:



The four colours encode four minimal slices of D at v :

- (A) **Red** : Both branches are retained; the assumption slice keeps x 's domain only and y 's codomain only (joining to $* \Rightarrow *$).
- (B) **Blue** : Symmetric to (A), taking the opposite choices for the assumption slice
- (C) **Green** : **Else**-branch omitted.
- (D) **Mulberry** : Symmetric to (C): **then**-branch omitted.

Slices (A) and (B) are minimal because each variable is sliced *differently* across the two uses, and none are *program* slices of each other. However, notice that (A) and (B) are not subslices of the product with x , where only 2 minima exist:



Aliasing. When this expression is embedded in the product (**if true then x else y , x**), the outer use of x *joins* onto the assumption slice. As slices (A) (B) (C) each also use x , the assumptions are aliased, and in slices (A) (B) the assumptions now available in the if statement are strictly greater, making the expression non-minimal.

Proposal. Use a stricter notion of synthesis slices that asks only “is the term irreducible under the full assumptions Γ ?”. Slices (A) and (B) are ruled out, but (C) and (D) are still valid. The next subsection makes this precise.

7.2 Term Slices

To rule out slices like (A) and (B) of Section 7.1 we strengthen the notion of slice: rather than allowing the assumption context to vary across $[\Gamma]$, we fix it at the full Γ and slice the term alone. The resulting slice is a sliced term that, under the original assumptions, still synthesises a type at least as precise as the query.

Definition 7.1 ( Term-minimal slice). A *term-minimal slice* of $D : \Delta; \Gamma \vdash e \uparrow \tau$ on $v \in [\tau]$, written $TermMin D \blacktriangleleft v$, is a term slice $\varsigma \in [e]$ such that $\Delta; \Gamma \vdash \varsigma \uparrow \psi$ for some $\psi \sqsubseteq v$ (with $\psi \in [\tau]$ by graduality, theorem 3.13).

7.3 Term-Minimality

Minimality lifts directly from the slice lattice on e alone:

Definition 7.2 ( Minimal Term-Minimal Slices). A $MinTermMin D \blacktriangleleft v$ is a $TermMin D \blacktriangleleft v$ minimal in $TermMin D \blacktriangleleft v$ under term-slice precision on $[e]$.

The two notions of minimality do not coincide: term-minimality is strictly stronger. Not all minimal slices are term-minimal under maximal assumptions.

Counterexample 7.3 ( Minimal $\not\Rightarrow$ Term-Minimal). Slices (A) and (B) of Section 7.1 are minimal synthesis slices but not term-minimal.

7.4 Existence of Minimal Term-Minimal Slices

As with synthesis slices, every term-minimal slice has a minimal slice below it:

Theorem 7.4 ( Existence of minimal term-minimal slices). For every $s : TermMin D \blacktriangleleft v$, there exists $m : MinTermMin D \blacktriangleleft v$ with $m \sqsubseteq s$.

The rest of this chapter develops a structural calculus that characterises minimal term slices via a judgement:

$$\Gamma \vdash e \uparrow \tau \blacktriangleleft v \rightsquigarrow \varsigma \uparrow \psi \dashv \gamma,$$

read “the typing $\Gamma \vdash e \uparrow \tau$ at query v produces term slice ς , synthesises ψ , and uses assumptions γ ”. The fourth output γ is a minimal assumption slice to retrieve a minimal slice corresponding to the term-minimal slice.

Term-Minimal $\rightarrow$ Minimal Slice. A minimal slice is then obtained by pairing ς with any minimal assumption context γ still synthesising a type at least as imprecise as v . This justifies the algorithmic strategy of the rest of the chapter: compute term-minimal slices structurally, then promote them to full minimal slices.

$$\boxed{\Delta; \Gamma \vdash e \uparrow \tau \blacktriangleleft v \rightsquigarrow \varsigma \uparrow \psi \dashv \gamma}$$

$$\begin{array}{c}
\uparrow\text{Var} \frac{\Gamma(x) = \tau \quad v \neq \square}{\Delta; \Gamma \vdash x \uparrow \tau \blacktriangleleft v \rightsquigarrow x \uparrow \tau \dashv x : v} \\
\uparrow\text{Gap} \frac{}{\Delta; \Gamma \vdash e \uparrow \tau \blacktriangleleft \square \rightsquigarrow \square \uparrow \square \dashv \emptyset} \\
\uparrow\text{Const} \frac{}{\Delta; \Gamma \vdash * \uparrow * \blacktriangleleft \top \rightsquigarrow * \uparrow * \dashv \emptyset} \\
\uparrow\text{Pair} \frac{\Delta; \Gamma \vdash e_1 \uparrow \tau_1 \blacktriangleleft v_1 \rightsquigarrow \varsigma_1 \uparrow \psi_1 \dashv \gamma_1 \quad \Delta; \Gamma \vdash e_2 \uparrow \tau_2 \blacktriangleleft v_2 \rightsquigarrow \varsigma_2 \uparrow \psi_2 \dashv \gamma_2}{\Delta; \Gamma \vdash (e_1, e_2) \uparrow \tau_1 \times \tau_2 \blacktriangleleft v_1 \times v_2 \rightsquigarrow (\varsigma_1, \varsigma_2) \uparrow \psi_1 \times \psi_2 \dashv \gamma_1 \sqcup \gamma_2} \\
\uparrow\text{Proj} \frac{v \neq \square \quad \Delta; \Gamma \vdash e \uparrow \tau_1 \times \tau_2 \blacktriangleleft \widehat{v}_i \rightsquigarrow \varsigma \uparrow \psi_1 \dashv \gamma}{\Delta; \Gamma \vdash \pi_i e \uparrow \tau_i \blacktriangleleft v \rightsquigarrow \pi_i \varsigma \uparrow \pi_i \psi_1 \dashv \gamma} \\
\uparrow\text{App} \frac{\Delta; \Gamma \vdash e_2 \downarrow \tau_1 \quad v \neq \square \quad \Delta; \Gamma \vdash e_1 \uparrow \tau_1 \Rightarrow \tau_2 \blacktriangleleft \square \Rightarrow v \rightsquigarrow \varsigma_{fn} \uparrow \psi_1 \dashv \gamma}{\Delta; \Gamma \vdash e_1 e_2 \uparrow \tau_2 \blacktriangleleft v \rightsquigarrow \varsigma_{fn}(\square) \uparrow \text{cod}(\psi_1) \dashv \gamma} \\
\uparrow\text{Lam} \frac{\Delta; \Gamma, x:\tau_1 \vdash e_2 \uparrow \tau_2 \blacktriangleleft v_2 \rightsquigarrow \varsigma_2 \uparrow \psi_2 \dashv \gamma, x:\phi_1 \quad \Delta; \Gamma, x:(\phi_1 \sqcup v_1) \vdash \varsigma_2 \uparrow \psi'_2}{\Delta; \Gamma \vdash \lambda x:\tau_1. e_2 \uparrow \tau_1 \Rightarrow \tau_2 \blacktriangleleft v_1 \Rightarrow v_2 \rightsquigarrow \lambda x:(\phi_1 \sqcup v_1). \varsigma_2 \uparrow (\phi_1 \sqcup v_1) \Rightarrow \psi'_2 \dashv \gamma} \\
\uparrow\text{Let} \frac{v_2 \neq \square \quad \Delta; \Gamma, x:\tau_1 \vdash e_2 \uparrow \tau_2 \blacktriangleleft v_2 \rightsquigarrow \varsigma_b \uparrow \psi_2 \dashv \gamma_2, x:v_1 \quad \Delta; \Gamma \vdash e_1 \uparrow \tau_1 \blacktriangleleft v_1 \rightsquigarrow \varsigma_d \uparrow \psi_1 \dashv \gamma_1 \quad \Delta; \Gamma, x:\psi_1 \vdash \varsigma_b \uparrow \psi'_2}{\Delta; \Gamma \vdash \text{let } x = e_1 \text{ in } e_2 \uparrow \tau_2 \blacktriangleleft v_2 \rightsquigarrow \text{let } x = \varsigma_d \text{ in } \varsigma_b \uparrow \psi'_2 \dashv \gamma_1 \sqcup \gamma_2} \\
\uparrow\text{TLam} \frac{\Delta, \alpha; \Gamma \vdash e \uparrow \tau \blacktriangleleft v \rightsquigarrow \varsigma \uparrow \psi' \dashv \gamma}{\Delta; \Gamma \vdash \Lambda \alpha. e \uparrow \forall \alpha. \tau \blacktriangleleft \forall \alpha. v \rightsquigarrow \Lambda \alpha. \varsigma \uparrow \forall \alpha. \psi' \dashv \gamma}
\end{array}$$

$$\boxed{\uparrow\text{TApp}: \text{ see Section 7.5.3.}}$$

$$\boxed{\uparrow\text{Case}: \text{ see Section 7.6.}}$$

Figure 21: Term-minimal slice calculus rules.

7.5 Calculating Term-Minimal Slices

As mentioned above, restricting to term-minimality allows products to be calculated by pairing sub-slices at the corresponding sub-queries, i.e. for $v_1 \times v_2$, slice the first component at v_1 , similarly for the second. More generally, if a typing rule has sub-terms which synthesise types, then recursively slice them. Otherwise, if a sub-term analyses against a type, it can be omitted entirely as it does not *provide* type information (only checking against existing type information).

The cases that deserve commentary are the binding constructs, function application, and type application; the $\uparrow\text{Lam}$: and $\uparrow\text{Let}$ rules each carry a body re-typing premise (rightmost) which re-derives the body's typing under the binder slice $\gamma(x)$.

7.5.1 Function Application

For example, for function application, omit the argument, and slice the function by $\square \Rightarrow v$, the argument type is not required to determine the return type:

$$\uparrow\text{App} \frac{\Delta; \Gamma \vdash e_2 \Downarrow \tau_1 \quad v \neq \square \quad \Delta; \Gamma \vdash e_1 \Uparrow \tau_1 \Rightarrow \tau_2 \blacktriangleleft \square \Rightarrow v \rightsquigarrow \varsigma_{fn} \Uparrow \psi_1 \dashv \gamma}{\Delta; \Gamma \vdash e_1 e_2 \Uparrow \tau_2 \blacktriangleleft v \rightsquigarrow \varsigma_{fn}(\square) \Uparrow \text{cod}(\psi_1) \dashv \gamma}$$

7.5.2 Lambdas, Lets, and Type Abstractions

Rules which bind variables are more complex, as they abstract a variable used in the minimised body. Slice them by first slicing the body, extracting the assumption it needed for the bound variable from γ , then slice the definition / type annotation using this. Notice that this is the *opposite* direction to typechecking.¹⁰ The corresponding rules are $\uparrow\text{Lam}$:, $\uparrow\text{Let}$, and $\uparrow\text{TLam}$.

7.5.3 Type Application

Type application works by matching the type function's type against the query to collect a list of constraints on the type variable, the *join* of which gives the sliced type argument $\hat{\sigma}$. The type function is then sliced with a matched query $\hat{v}$, where each α -position of the query is replaced by α and non- α positions are carried through unchanged. Figure 22 illustrates the construction on a query whose two α -positions give differing constraints.

$e \Uparrow \forall \alpha. (\alpha \times \alpha \times \mathbf{Bool})$, applied at $\sigma = * \Rightarrow (\mathbf{Bool} \times \mathbf{Bool})$:

So $e\langle\sigma\rangle \Uparrow (* \Rightarrow (\mathbf{Bool} \times \mathbf{Bool})) \times (* \Rightarrow (\mathbf{Bool} \times \mathbf{Bool})) \times \mathbf{Bool}$.

$$\text{Query } v = \underbrace{* \Rightarrow \square}_{\alpha} \times \underbrace{\square \Rightarrow (\mathbf{Bool} \times \square)}_{\alpha} \times \square$$

$$\text{Matched query } \hat{v} = \alpha \times \alpha \times \square$$

$$\text{Constraints } * \Rightarrow \square \sqsubseteq \alpha \quad \square \Rightarrow (\mathbf{Bool} \times \square) \sqsubseteq \alpha$$

$$\text{Constraints Join } \hat{\sigma} = * \Rightarrow (\mathbf{Bool} \times \square)$$

Figure 22: Type application.

¹⁰ e.g. let bindings type the definition first, then use the result to type the body.

7.6 $\triangle$ Fixed Point Algorithm for Case Expressions

This is the complex case, minimal slices must have no redundant type information from both branches, so the query must be split between the branches. However, simply splitting the query disjointly between the branches is not enough to ensure minimality.

The Overlap Problem. Suppose a case expression **case** e **of** $x. e_1 \mid y. e_2$ synthesises $\tau_1 \sqcup \tau_2$. Suppose we disjointly split a query $v \in \lfloor \tau_1 \sqcup \tau_2$ into $v_1 \in \lfloor \tau_1$ and $v_2 \in \lfloor \tau_2$. Then slice the branches, but as minimal slices are not necessarily exact (theorem 4.3), then they may synthesise strictly more precise ψ_1 and ψ_2 . In this situation, branch 1 actually only needed to cover what ψ_2 didn't cover, which is strictly less than the v_1 it was queried under, so it may not be minimal on this, and vice versa for branch 2.



Figure 23: Abstract View of Overlap and Residuals.

7.6.1 Branch Residuals

When exactly are two branch slices jointly minimal? Let v_1 and v_2 be the branch queries, then when (see Figure 23b):

- $v_1 \sqsubseteq v \setminus \psi_2$. Branch 1 is queried on not more than is *actually* provided by branch 2 or contained within v .
- $v_2 \sqsubseteq v \setminus \psi_1$. Branch 2 is queried on not more than is *actually* provided by branch 1 or contained within v .
- $v \sqsubseteq \psi_1 \sqcup \psi_2$. The joined branch types actually cover the query.

It is still possible for ψ_1 and ψ_2 to overlap, but any overlap is not contained in the queries, so the branches are still minimal.

Iteration. However, these are *mutually dependent* slices, so they need iteration to solve. Let v_1^i be the query on the i th iteration, (σ_1^i, ψ_1^i) be the resulting slice and similarly for branch 2. Then iterate setting $v_1^1 = v \setminus \psi_2^0$, then setting $v_2^1 = v \setminus \psi_1^1$. Starting at $(\psi_1^0, \psi_2^0) = (\top, \square)$. We wish to show that ψ_1 decreases monotonically and ψ_2 increases monotonically.

As co-Heyting subtraction is anti-monotone in its second argument (theorem 2.1), then as ψ_2 increases,¹¹ v_1 decreases and as ψ_1 decreases, v_2 increases. At the start, $\psi_2^0 = \square$, so ψ_2 increases on the first step, so v_1 decreases on the first step.

By monotonicity of minimum synthesis slices (theorem 4.8), we can find σ_1 decreasing and hence ψ_1 decreasing. Hence, v_2 increases (by anti-monotonicity). However, an upwards monotonicity theorem, which would conclude the next step, that ψ_2 monotonically

¹¹ All use of increase/decrease here is in the monotonic sense.

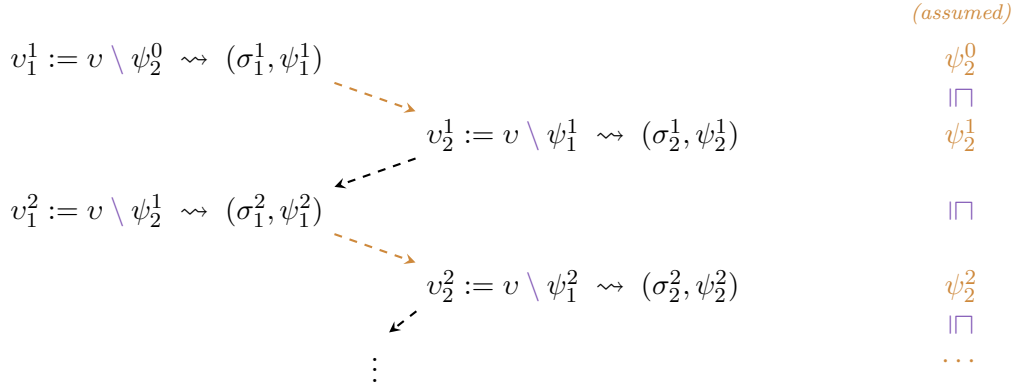


Figure 24: Branch Fixed Point Iteration

increases, does *not* hold (theorem 7.5). But, when it does, v_1 would decrease inductively, completing the loop.

Hence, *when the chain ψ_2 monotonically increases*, then ψ_1 monotonically decreases. So, by Kleene's finite fixed point theorem (theorem 2.2) this iteration leads to a fixed point with $v_1 = v \setminus \psi_2$ and $v_2 = v \setminus \psi_1$, satisfying the first two constraints for jointly minimal branches. The third condition $v \sqsubseteq \psi_1 \sqcup \psi_2$ then follows from validity of the branch slices ($v_1 \sqsubseteq \psi_1$):

$$\begin{aligned}
 v &\sqsubseteq \psi_2 \sqcup (v \setminus \psi_2) && \text{(co-Heyting: } a \sqsubseteq b \sqcup (a \setminus b) \text{)} \\
 &= \psi_2 \sqcup v_1 && \text{(definition of } v_1 \text{)} \\
 &\sqsubseteq \psi_2 \sqcup \psi_1 && \text{(by } v_1 \sqsubseteq \psi_1 \text{).}
 \end{aligned}$$

Counterexample 7.5 (Upwards Non-Monotonicity of Term-Minimal Slices). *Fix $\Gamma = (z : * \times *)$ and $D : \Delta; \Gamma \vdash \text{if } \square \text{ then } z \text{ else } (*, \square) \uparrow * \times *$. The less precise query $v' = * \times \square$ admits the following term-minimal slice:*

$$\begin{array}{c}
 \text{assumptions} \\
 \boxed{z : * \times *} \\
 \text{if } \square \text{ then } \square \text{ else } (*, \square)
 \end{array}$$

*The more precise query $v = * \times *$, by contrast, has a unique term-minimal slice – synthesising $* \times *$ forces use of z :*

$$\begin{array}{c}
 \text{assumptions} \\
 \boxed{z : * \times *} \\
 \text{if } \square \text{ then } z \text{ else } \square
 \end{array}$$

The two are incomparable.

Therefore, this part of the algorithm is partial. In reality, this covers most cases, and you can revert to the brute force algorithm otherwise. Devising a complete (non-brute force) algorithm is left to future work. Another potential downside of this algorithm is that it is *biased*, it prefers to select typing information from branch 1.

7.6.2 Scrutinee Descent

The branches are now jointly minimal under the full context. But, case statements also include *bindings*, whose type are sourced by another term. However, unlike with let bindings, we can't simply take the minimal assumptions of the branches, because those may only provide enough to synthesise the residual queries v_i , which may not cover the original query v .¹²

As case scrutinee terms are typically small in real world code (e.g. if statements), we can simply do a brute-force descent on the scrutinee. But, if the term is significantly larger than the type, it makes sense to can perform a brute-force descent on the bound types to find a minimal pair of assumptions to cover the query v : that is, $x : \phi_1; \Gamma \vdash \sigma_1 \vdash \psi'_1$ and $y : \phi_2; \Gamma \vdash \sigma_2 \vdash \psi'_2$ with $v \sqsubseteq \psi'_1 \sqcup \psi'_2$; this can be done by descending upon the type (hence why this is only useful when the type is smaller than the term). Then slice the scrutinee according to $\phi_1 + \phi_2$ and use this as a smaller starting point to descend on.

○ **Optimisation:** However, if $v \sqsubseteq v_1 \sqcup v_2$ already, then you can do the same as the let binding case: slice the scrutinee according to the (two) minimal assumptions required for the branch bindings, and derive the minimal assumptions by joining the scrutinee and branches minimal assumptions.

7.6.3 The Assumption Slice

At this point we have a term-minimal slice, but we still need a minimal assumption slice. Again, this does not directly follow from the minimal assumptions for the branches individually, due to *aliasing* of common assumptions. However, you can join these individual minima alongside the minimal assumptions for the scrutinee (calculated by brute-force¹³ or over-approximated using the slice on $\phi_1 + \phi_2$). This gives a set of valid assumptions to synthesise v which will be *near-minimum* and work as a good starting point to brute-force from.

7.7 △ Calculating Analysis Slices

Analysis slices are calculated directly from synthesis slices. The only rule that creates a type checking judgement is function application. Consider the analysis slice of a function argument on v_2 ; it is just the function application and a slice of the function under $\square \rightarrow v_2$.

The key difference is that analysis slices are *constructed from the focus upwards*. An unannotated function is an *inner* (theorem 5.2) analysis slice; so it defers calculation of its outer context by returning a minimal analysis type v_{outer} to impose upon the unannotated function which still imposes the original query, v , on the focus, i.e. $v_{outer} \triangleq \square \rightarrow v$. The other analysis rules are similar. The only additional complexity is for binding constructs which also must slice the bindings according to the assumptions required for the inner analysis slice.

¹² We only have that $v \sqsubseteq \psi_1 \sqcup \psi_2$.

¹³ *Ascending* from $\emptyset$ being more efficient here.

8 $\triangle$ Minimum-Size Slicing

Legend: $\bigcirc$ fully mechanised, $\otimes$ mechanised with postulates, $\triangle$ not mechanised, $\square$ important theorem, $\square$ counterexample.

We can calculate a *minimal* synthesis slice in polynomial time (section 4.3). However, minimal slices are not unique, so a natural extension is to rank them by *size* and ask for a minimal slice of *minimum size*. This chapter reduces the *set cover optimisation problem* to calculating minimum size slice, meaning minimum-size slicing is **NP-hard**. I (informally) prove this for term-minimal synthesis slices, and show that the argument extends to general minimal synthesis slices; I expect, but do not show, that it extends further to analysis slices, whose non-determinism is wholly inherited from synthesis (section 5.2).

8.1 $\triangle$ Definitions

Definition 8.1 (Size of an Expression Slice). For an expression slice $\varsigma \in [e]$, the size $|\varsigma|$ counts non- $\square$ syntactic positions:

$$|\square| = 0, \quad |*| = 1, \quad |x| = 1, \quad |c(\varsigma_1, \dots, \varsigma_k)| = 1 + \sum_i |\varsigma_i|.$$

Definition 8.2 (Minimum-Size TermMinimal Slice Problem (Min-TM-Slice)). Given a derivation $D : \Delta; \Gamma \vdash e \uparrow \tau$ and query $v \in [\tau]$, the minimum-size term-minimal slice problem is to compute

$$\text{minSize}(D, v) \triangleq \min\{|\sigma| : \sigma \in \text{TermMin } D \blacktriangleleft v\}.$$

Definition 8.3 (Minimum Set Cover (Optimisation) Problem (Min-Set-Cover)). Given a universe $U = \{1, \dots, n\}$ and a family $S = S_1, \dots, S_m \subseteq U$, compute

$$K^*(U, S) \triangleq \min\{|J| : J \subseteq \{1, \dots, m\}, \bigcup_{j \in J} S_j = U\}.$$

The minimum set cover optimisation problem is NP-hard [19, §16.1, p. 414], hence any polynomial reduction from it, is also NP-hard.

8.2 $\triangle$ The reduction

Definition 8.4 (Reduction from Min-set-cover). Given an instance $(U, S_1, \dots, S_m)$ of Min-Set-Cover, suppose wlog. $|U| = n$ and $\bigcup_j S_j = U$. Then construct $D : \Delta; \Gamma \vdash e \uparrow \tau$ and query $v = \tau$ as follows:

- **Target type τ :** $\tau = \underbrace{* \times (* \times (\dots \times (* \times *)))}_{n \text{ projections}}$. Position i of τ corresponds to element i of U .
- **Encoding $S_j \mapsto T_j$:** For each j , construct type slice family $T_j \in [\tau]$ by omitting all projections i for $i \notin S_j$. Then $\bigsqcup_j T_j = \tau$ iff $\bigcup_j S_j = U$, where $\bigsqcup_j$ is the finite join of family T_j .
- **Assumption context Γ :** $\Gamma = (x_1 : T_1, \dots, x_m : T_m)$, so each x_j synthesises T_j exactly under $\uparrow \text{Var}$.

- **Let-chain leaves ℓ_j :** For each j , define a chain of $m - 1$ let-bindings that thread x_j through fresh names:

$$\ell_j \triangleq \text{let } n_1 = x_j \text{ in let } n_2 = n_1 \text{ in } \cdots \text{let } n_{m-1} = n_{m-2} \text{ in } n_{m-1}$$

ℓ_j synthesises T_j by repeated application of $\uparrow\text{Let}$, and has size $2m - 1$. Any strict sub-slice of ℓ_j breaks the chain and synthesises only $\square$, so ℓ_j behaves as an all-or-nothing unit.

- **Case tree e :** Construct any binary case tree with leaves $\ell_1, \dots, \ell_m$, scrutinees $\square$, and binders x .

By theorem 3.4 the leaves' synthesised types are mutually consistent, so the case rule applies at every node, giving $\Delta; \Gamma \vdash e \uparrow \bigsqcup_j T_j = \Delta; \Gamma \vdash e \uparrow \tau$. The tree has size $|e| = m(2m - 1) + (m - 1) = 2m^2 - 1$.

- **Query:** $v = \tau$.

This construction is trivially computable in polynomial time, $\mathcal{O}(n + m^2 + \sum_j |S_j|)$ (quadratic in m because of the per-leaf let chains).

Concretely, view each set S_j as a *variable* x_j whose type is a slice of the target carrying $*$ exactly where S_j has an element. The chain ℓ_j at each leaf makes retaining x_j in the slice cost $2m - 1$ (the full chain) and omitting it cost 0, for minimal slices.¹⁴ The per-leaf cost $2m - 1$ strictly dominates the (at most) $(m - 1)$ variation in retained case nodes, so the minimum-size slice agrees with the minimum set cover. Figure 25 draws this correspondence on a 4-element universe.

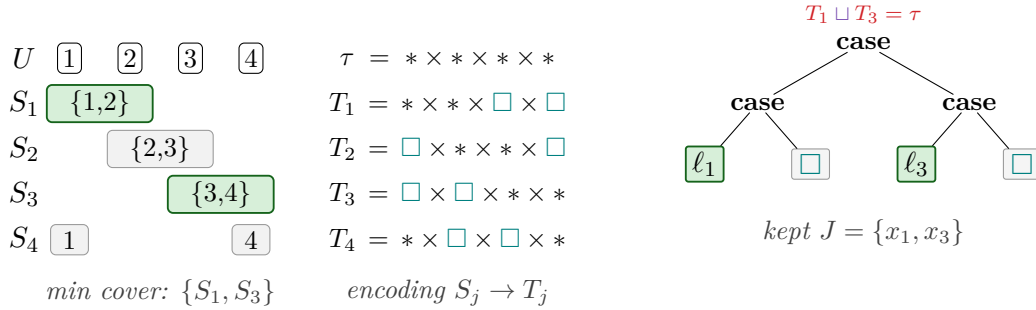


Figure 25: Set cover (left), the encoding (middle), and the induced case-tree slice (right), with a minimal set cover selected.

8.3 $\triangle$ Correctness

Throughout this subsection, write $J(\sigma) = \{j : \ell_j \text{ is fully retained in } \sigma\}$ for the set of leaves whose entire let-chain is kept.

Lemma 8.5 (Validity is Set-Cover). *A term-minimal slice σ of D at $v = \tau$ is valid iff $J(\sigma)$ is a set cover of U .*

Proof. A fully retained chain ℓ_j synthesises T_j ; any strict sub-slice of ℓ_j (or a fully gapped leaf) synthesises $\square$. Walking up the case tree, the case rule joins branch types pointwise, so σ synthesises $\bigsqcup_{j \in J(\sigma)} T_j$. Validity ($\sqsubseteq \tau$) holds iff every position of τ is covered: iff $\bigcup_{j \in J(\sigma)} S_j = U$. $\square$

¹⁴ Partially slicing any part of the chain is never minimal.

Lemma 8.6 (Minimum Slice Size). *For (Γ, e, τ) encoding Min-Set-Cover problem (U, S) , every minimum-size valid slice σ has $|J(\sigma)| = K^*(U, S)$. Hence $K^*(U, S)$ is recovered from such a σ by counting retained chains, in time $\mathcal{O}(|\sigma|) = \mathcal{O}(m^2)$.*

Proof. A minimum-size term-minimal slice σ' is associated with the set $J(\sigma')$ by counting the retained leaves. This requires a single loop over the case tree, whose size is quadratic in m , so $J(\sigma')$ is computable in $\mathcal{O}(m^2)$ time.

Any minimal slice σ' has size $|J(\sigma')| \cdot (2m-1) + k$ where $0 < k < m$ counts the number of case statements in the slice. Therefore, suppose σ' corresponds to a non-minimal solution to set-cover, then by theorem 8.5, some σ has $|J(\sigma)| < |J(\sigma')|$, hence:

$$\begin{aligned} |\sigma| &< |J(\sigma)| \cdot (2m-1) + m && \text{(bound on case-node count)} \\ &\leq (|J(\sigma')| - 1) \cdot (2m-1) + m && \text{(by } |J(\sigma)| < |J(\sigma')| \text{)} \\ &= |J(\sigma')| \cdot (2m-1) - m + 1 && \text{(arithmetic)} \\ &\leq |\sigma'| && \text{(bound on case-node count).} \end{aligned}$$

Hence, σ' would not be a minimum-size slice; by contradiction, a minimum-sized σ' must therefore correspond to a minimum set cover. $\square$

Theorem 8.7 (Minimum-Size Term-Minimal-Slicing is NP-hard). *Computing $\text{minSize}(D, v)$ is NP-hard.*

Proof. Definition 8.4 is a polynomial-time reduction from Min-Set-Cover to Min-TM-Slice with Lemma 8.6 recovering $K^*(U, S)$ from $\text{minSize}(D, v)$ in polynomial time. Hence, NP-hardness of Min-Set-Cover [19, §16.1, p. 414] transfers onto Min-TM-Slice. $\square$

8.4 $\triangle$ Extension to Minimal Synthesis Slices

The reduction above can be embedded also into minimum synthesis slices in general, by forcing the minimal assumptions slice to be full by producing the case tree with each of the variables x_j and querying each set in this product fully. This forces every variable to be used fully in the assumption slice.

Definition 8.8 (Reduction to Min-Syn-Slice). Let (Γ, e_0, τ) be the term-minimal encoding of Definition 8.4. Form the product expression and query

$$e \triangleq (e_0, x_1, x_2, \dots, x_m), \quad v \triangleq \tau \times T_1 \times T_2 \times \dots \times T_m,$$

under the same assumption context $\Gamma = (x_1 : T_1, \dots, x_m : T_m)$.

Lemma 8.9 (Maximal Assumption Slice). *For every valid slice $\rho = (\gamma, \varsigma)$ of Δ ; $\Gamma \vdash e \uparrow v$ at v , $\gamma = \Gamma$.*

Proof. The $(j+1)$ -th projection is x_j and is queried at T_j . This projection must synthesise T_j under γ via $\uparrow\text{Var}$. Hence, we require $\gamma(x_j) = T_j$, so $\gamma = \Gamma$. $\square$

Theorem 8.10 (Minimum-Size Syn-Slice is NP-hard). *Computing $\min\{|\rho| : \rho \in \text{SynSlice } D \blacktriangleleft v\}$ is NP-hard.*

Proof. By Lemma 8.9, $|\gamma| = |\Gamma|$ is fixed by the input, and the outer-tuple variable positions $x_1, \dots, x_m$ contribute a further fixed m . Hence $|\rho| = |\Gamma| + m + |\varsigma_0|$, where ς_0 slices the case-tree component e_0 , and minimising $|\rho|$ reduces to the term-minimal problem on (e_0, τ) . The construction is polynomial in $n+m$, and NP-hardness transfers from Theorem 8.7. $\square$

9 Relation to Other Forms of Slicing

Program slicing of various different forms, relating to dynamic execution, data dependencies, and type errors, has been studied extensively. In this chapter I compare type slicing with three other program slicing traditions: classical *program slicing* [35, 32], *Galois slicing* [23], and *type error slicing* [34, 15]. Type slicing differs from all three, but has some similarities to Galois slicing.

9.1 Classical Program Slicing

Classical program slicing [35, 32] was developed for imperative programs. Slices are syntactically obtained by deleting lines in the code, which is less flexible than omitting arbitrary sub-terms in a syntax tree as used here. The equivalent of type queries is a ‘slicing criterion’, which is a (program-point, variable) pair, with the slice preserving the runtime value of the chosen variable at the chosen point. Type queries have a more complex, compound structure as compared to simple variables, and slices cannot preserve the type query in general (theorem 4.3), so we adopt the notion of *validity* instead of preservation.

Program slicing has static [35] and dynamic [18] variants, depending on whether the slicing criterion is preserved over *all* inputs to the program or a particular execution. As typing is deterministic (theorem 3.17), this distinction does not apply to type slicing.

9.2 Galois Slicing

Galois slicing [23] is a lattice-theoretic version of type slicing that has subsequently been applied and related to a wide range of calculi and user interfaces [29, 22, 24, 3].

Type slicing is related in the use of lattices and precision. However, Galois slices define a Galois connection, which unfortunately does not exist for type slicing.

9.2.1 Background: Galois connections and the adjoint functor theorem

For context, an order-theoretic Galois connection is defined as follows:

Definition 9.1 (Galois connection). Let $(P, \sqsubseteq_P)$ and $(Q, \sqsubseteq_Q)$ be posets. A pair of monotone maps $f : P \rightarrow Q$ and $g : Q \rightarrow P$ is a *Galois connection*, written $f \dashv g$ with f the lower adjoint and g the upper adjoint, when for every $p \in P$ and $q \in Q$

$$f(p) \sqsubseteq_Q q \iff p \sqsubseteq_P g(q).$$

Proposition 9.2 (Adjoint Functor Theorem for Complete Lattices [13, Prop. 7.34]). *Let P, Q be complete lattices. A monotone map $g : Q \rightarrow P$ is the upper adjoint of a (necessarily unique) Galois connection if and only if g preserves all meets. When g preserves meets, the lower adjoint f is given pointwise by*

$$f(p) = \bigcap \{ q \in Q \mid g(q) \sqsupseteq p \}.$$

9.2.2 Slicing

Perera et al. work in a functional setting in which a program e is evaluated under an environment ρ to a value v , written $\rho, e \Downarrow v$. For a fixed such terminating execution, they

form the slice lattice $[\rho, e]$ of partial programs below (ρ, e) and the slice lattice $[v]$ of partial values below v (Perera et al. refer to these as prefixes, $\text{Prefix}(\rho, e)$ and $\text{Prefix}(v)$). Type slices have the same shape as Perera’s slices: a partial program (here the slice ρ) paired with a partial value (here the synthesised type ψ), mediated by a derivation (here the typing derivation).

Restricted to the slice lattice below the executed program, evaluation lifts to a function

$$\text{eval}_{\rho,e} : [\rho, e] \rightarrow [v]$$

which is total, monotone, and crucially meet-preserving:

$$\text{eval}_{\rho,e}(p_1 \sqcap p_2) = \text{eval}_{\rho,e}(p_1) \sqcap \text{eval}_{\rho,e}(p_2).$$

By the adjoint functor theorem, $\text{eval}_{\rho,e}$ is then the upper adjoint of a unique Galois connection, with lower adjoint

$$\text{uneval}_{\rho,e} : [v] \rightarrow [\rho, e], \quad \text{uneval}_{\rho,e}(u) = \bigsqcap \{ p \in [\rho, e] \mid \text{eval}_{\rho,e}(p) \sqsupseteq u \}$$

This $\text{uneval}_{\rho,e}$ is the ‘backward slice function’: it sends a partial-value query u to the meet of all slices whose evaluation suffices to produce u , and that meet is itself in the set, so $\text{uneval}_{\rho,e}(u)$ is a least slice for u .

9.2.3 Failure of Meet-Preservation for Type Synthesis

The natural type-slicing analogue of $\text{eval}_{\rho,e}$ is the synthesis function

$$\phi_D : [\Gamma, e] \rightarrow [\tau]$$

which, given a slice $\rho \sqsubseteq (\Gamma, e)$, returns the unique synthesised type of ρ under D . The function is well-defined by syn-unicity, monotone by syn-precision, and exists for any derivation D .

However, the forward map ϕ_D is not meet-preserving in general. The case rule witnesses this: consider a derivation

$$D : \Delta; \Gamma \vdash (\text{case } e \text{ of } x_1. e_1 \mid x_2. e_2) \uparrow \tau$$

in which both branches independently synthesise the same non-gap type τ . The slice s_1 keeping only the first branch and the slice s_2 keeping only the second each yield $\phi_D = \tau$, but their meet $s_1 \sqcap s_2$ omits both branches, giving $\phi_D(s_1 \sqcap s_2) = \square \sqsubseteq \tau = \phi_D(s_1) \sqcap \phi_D(s_2)$.

9.3 Type Error Slicing

Type error slicing, like type slicing, is a static analysis on a typing problem. Both share the broad goal of attributing type-level information back to source positions, but differ in language target, the underlying mathematical structure, and the scope.

Wand [34] and Haack and Wells [15] work in globally inferred Hindley-Milner: the program is reduced to a system of equality constraints over type variables, and a type error slice is computed as a minimal unsatisfiable subset of the constraint set, projected back to the corresponding minimal subset of source positions.

These program slices are not structure preserving programs, they are flat sets of constraints and sources, rather than actual programs. This means the results themselves cannot be type checker nor can they be executed.

There is also a difference in scope. Type error slicing answers “*why does this program fail to type-check?*”, whereas type slicing answers “*why does this expression have this type?*”. Via marking (section 6) [37], which elaborates every well-formed expression into a totally typed derivation, type slicing applies also to ill-typed programs, so “why this type?” subsumes “why this fails to type-check?” as the special case of explaining the type of a marked expression.

10 Further Extensions & Applications

10.1 Further Extensions

10.1.1 Implicit Polymorphism

Globally inferred languages typically have much more confusing typing derivations and type errors than locally inferred languages. Extending the calculus to implicit polymorphism in a bidirectional setting would expand the application significantly. In particular, the ability to explain even well-typed code is particularly useful here as error marks are often localised to the wrong location, resulting the real error being in well-typed code. Adapting the constraint slicing systems of Wand [34] and Haack and Wells [15] to integrate with type slicing is worth exploring.

10.1.2 Set Theoretic Types

Set theoretic types extend the type language with union, intersection, and negation. A term can then carry several types simultaneously, allowing typing of overloaded functions, and typecase constructs, commonly used in real world hybrid typed languages (e.g. TypeScript, or Elixir [10]). These types could be added to extend the slice lattices of this paper (section 3.2), and type slicing could be used to explain the sources of union types by omitting appropriate branches (including typecase structures); similarly, irrelevant overloads to functions would be omitted.

Figure 3 demonstrated a simple example of this by inferring sum types using dynamic types, in much the same way as simple union types would be inferred over case statements.

Gradual set theoretic types have been explored by Castagna [11, 12], and more recently, Ângelo’s recent thesis [2] establishes graduality specifically for intersection types with a proof of graduality.

10.1.3 Occurrence Typing

Occurrence typing [33] is another common technique in typing flexible or hybrid type systems. Inside the true branch of `if (typeof x === int) ... else ...`, the type of `x` is inferred as an `int`. These types are similarly difficult to reason about, especially in large code, and, unlike the constraint-based type error slicing surveyed in section 9.3, do genuinely require a syntactically well-formed program slice to reason about the affect of data-flow on the types. Therefore, type slicing is a particularly good, and sufficiently flexible, way to explain types in such a system. However, to my knowledge there is no existing formal research of occurrence typing in a bidirectional setting.

10.2 Further Applications

10.2.1 Type-Modifying Refactorings

Refactoring a term such that its type changes would typically scatter type errors across the refactored locations. Type slicing can be used to semi-automatically fix these type errors.

Many locations to refactor will have syntactically different surrounding contexts, but may have the identical analysis contribution slices (extending contribution slice from

section 4.6 to the analogous construct on minimal analysis slices).¹⁵ We could treat these slices as *templates* which we define a refactor for, that performs edits to the elements in the minimal slice itself, possibly moving, but not editing, entire folded terms.

These edits could be semi-automated by referring back to the user to fix each template, or otherwise fully automatic by using dedicated solvers or LLMs. In particular, the slices provide only the necessary context to an LLM, restricting edits to code regions actually relevant to the type fixes. LLM edits on incomplete programs with holes is under active research [6] in the Hazel language [20], which this core calculus (section 3) is very similar to.

Figure 26 shows two syntactically distinct usages of a refactored `helper` term who both collapse to the same minimal analysis slice. An edit on that template (e.g. replacing `Int` with `String`) fixes both sites at once.

Changing these annotations will then lead to errors at other use sites not involved in the initial refactor, and we can treat synthesis contribution slices as templates to change in much the same way. By treating these edits as new refactorings, we would loop, fixing errors at each step until none are left (or if making no progress).

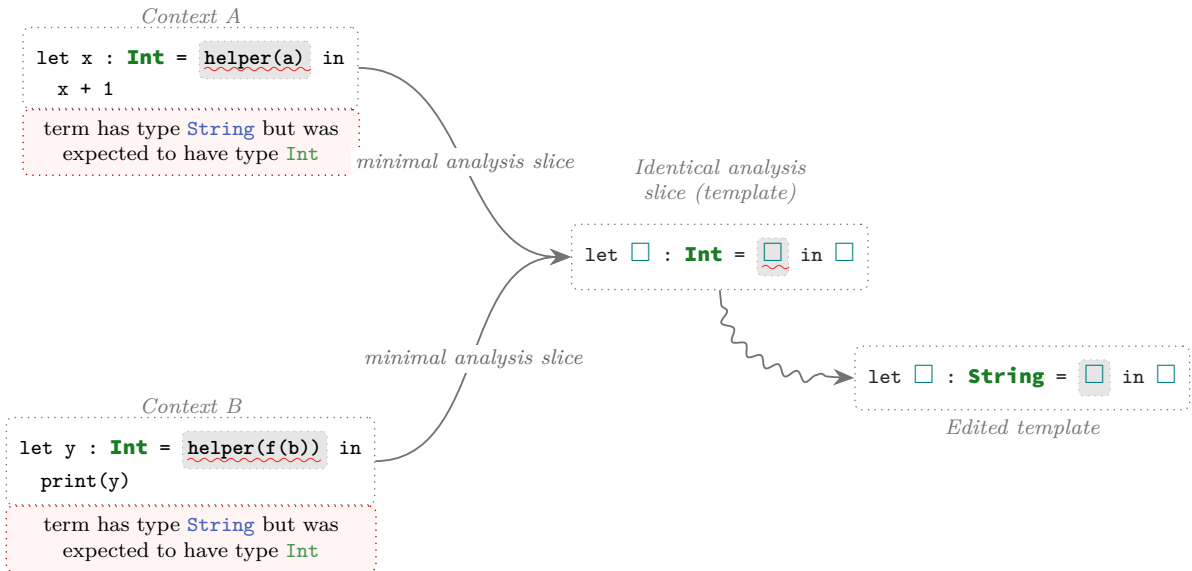


Figure 26: Two distinct contexts with the same analysis slice, performing the same (initial) fix.

10.2.2 Cast Slicing & Dynamics

A gradually-typed language elaborates source programs into a cast calculus and runs the resulting casts dynamically. Type slices can be attached directly to those casts. Hence, casts can then explain why they were inserted by pointing back to code; the source will be a slice of the term being cast, and the target a slice of its context. Approximate minimal slices can be tagged directly onto types allowing them to be decomposed at runtime, allowing dynamic casts to point back to code. This is particularly useful for debugging dynamic type errors, which do not typically point back to code, but do when carrying type slices. However, the strong properties of slices being slices of the original program is not maintained during dynamic execution. Tracking dynamic execution, perhaps integrating

¹⁵ Formally, we would compare their *minimally scoped* slices, in the sense of the minimally scoped checking contexts in [9].

ideas from Perera [23, 22], would allow users to trace back casts to source code with stronger guarantees. This application is detailed thoroughly in Carroll [8].

11 Conclusions

This dissertation presents the type slicing theory for a bidirectional calculus with explicit System F polymorphism, bindings, products, and sums, mechanising a large majority of the theory in Agda. A more rigorous account of typing assumptions involved in slices slices was considered with extensive exploration of provably correct algorithms for calculating type slices, including various optimisations.

The theory is formally integrated with the error-marking calculus of Zhao et al. [37], formally connecting it to ill-typed programs, justifying its *sound* use in debugging *all* static type errors of a program.

Type slicing was discussed to be substantially different from previous program slicing techniques, of which there are several varieties, none of which being directly applicable to the problem of explaining types in bidirectional type systems. Finally, further extensions and applications were proposed, which would place type slicing as a promising tool for use in practical gradually typed languages after extensions, both for explaining static *and dynamic* types and for code refactoring.

References

- [1] Abdulaziz Alaboudi and Thomas D. LaToza. *An Exploratory Study of Debugging Episodes*. 2021. arXiv: [2105.02162](https://arxiv.org/abs/2105.02162) [cs.SE]. url: <https://arxiv.org/abs/2105.02162>.
- [2] Pedro Jorge Fernandes Ângelo. “Gradual Intersection Types”. PhD thesis. Faculdade de Ciências, Universidade do Porto, 2024.
- [3] Robert Atkey and Roly Perera. *Galois Slicing as Automatic Differentiation*. arXiv preprint arXiv:2511.09203. 2025.
- [4] Moritz Beller et al. “On the dichotomy of debugging behavior among programmers”. In: *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. Association for Computing Machinery, 2018, pp. 572–583. doi: [10.1145/3180155.3180175](https://doi.org/10.1145/3180155.3180175).
- [5] Garrett Birkhoff. *Lattice Theory*. Vol. 25. Colloquium Publications. American Mathematical Society, 1940. doi: [10.1090/coll/025](https://doi.org/10.1090/coll/025).
- [6] Andrew Blinn et al. “Statically Contextualizing Large Language Models with Typed Holes”. In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA2 (Oct. 2024). issn: 2475-1421. doi: [10.1145/3689728](https://doi.org/10.1145/3689728). url: <http://dx.doi.org/10.1145/3689728>.
- [7] Maurice Bruynooghe. *Adding redundancy to obtain more reliable and more readable prolog programs*. Tech. rep. Department of Computer Science, KU Leuven, Leuven, Belgium, 1982.
- [8] Max Carroll. *Type error debugging in Hazel*. Tech. rep. UCAM-CL-TR-XXX. Originally a Part II dissertation, University of Cambridge. Hazel implementation branch: `witnesses-type-slicing`. University of Cambridge, Computer Laboratory, 2025.
- [9] Max Carroll, Anil Madhavapeddy, and Patrick Ferris. *Decomposable Type Highlighting for Bidirectional Type and Cast Systems*. Presented at the Workshop on Human Aspects of Types and Reasoning Assistants (HATRA), ICFP/SPLASH. 2025. url: https://maxcarroll0.github.io/assets/papers/Carroll-Decomposable_Type_Highlighting.pdf.
- [10] Giuseppe Castagna, Guillaume Duboc, and José Valim. “The Design Principles of the Elixir Type System”. In: *The Art, Science, and Engineering of Programming* 8.2 (2024), 4:1–4:34. doi: [10.22152/programming-journal.org/2024/8/4](https://doi.org/10.22152/programming-journal.org/2024/8/4).
- [11] Giuseppe Castagna and Victor Lanvin. “Gradual Typing with Union and Intersection Types”. In: *Proceedings of the ACM on Programming Languages* 1.ICFP (2017), 41:1–41:28. doi: [10.1145/3110285](https://doi.org/10.1145/3110285).
- [12] Giuseppe Castagna et al. “Gradual Typing: A New Perspective”. In: *Proceedings of the ACM on Programming Languages* 3.POPL (2019), 16:1–16:32. doi: [10.1145/3290329](https://doi.org/10.1145/3290329).
- [13] Brian A. Davey and Hilary A. Priestley. *Introduction to Lattices and Order*. 2nd. Cambridge University Press, 2002. doi: [10.1017/CB09780511809088](https://doi.org/10.1017/CB09780511809088).
- [14] Jana Dunfield and Neelakantan R. Krishnaswami. “Bidirectional Typing”. In: *ACM Computing Surveys* 54.5 (2022), 98:1–98:38. doi: [10.1145/3450952](https://doi.org/10.1145/3450952).

- [15] Christian Haack and J. B. Wells. “Type Error Slicing in Implicitly Typed Higher-Order Languages”. In: *Programming Languages and Systems (ESOP)*. Vol. 2618. LNCS. Springer, 2003, pp. 284–301. doi: [10.1007/3-540-36575-3_20](https://doi.org/10.1007/3-540-36575-3_20).
- [16] Robert Harper. *Practical Foundations for Programming Languages*. 2nd ed. Cambridge University Press, 2016. isbn: 9781107150300. doi: [10.1017/cbo9781316576892](https://doi.org/10.1017/cbo9781316576892).
- [17] Gérard Huet. “The Zipper”. In: *Journal of Functional Programming* 7.5 (1997), pp. 549–554. doi: [10.1017/S0956796897002864](https://doi.org/10.1017/S0956796897002864).
- [18] Bogdan Korel and Janusz Laski. “Dynamic Program Slicing”. In: *Information Processing Letters* 29.3 (1988), pp. 155–163. doi: [10.1016/0020-0190\(88\)90054-3](https://doi.org/10.1016/0020-0190(88)90054-3).
- [19] Bernhard Korte and Jens Vygen. *Combinatorial Optimization: Theory and Algorithms*. 6th ed. Vol. 21. Algorithms and Combinatorics. See §16.1 (p. 414) for NP-hardness of Minimum Set Cover. Springer, 2018. doi: [10.1007/978-3-662-56039-6](https://doi.org/10.1007/978-3-662-56039-6).
- [20] Cyrus Omar et al. “Live functional programming with typed holes”. In: *Proceedings of the ACM on Programming Languages* 3.POPL (Jan. 2019), pp. 1–32. issn: 2475-1421. doi: [10.1145/3290327](https://doi.org/10.1145/3290327). url: <http://dx.doi.org/10.1145/3290327>.
- [21] John-Paul Ore et al. “Assessing the Type Annotation Burden”. In: *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. ASE 2018. ACM, 2018, pp. 190–201. doi: [10.1145/3238147.3238173](https://doi.org/10.1145/3238147.3238173).
- [22] Roly Perera, Deepak Garg, and James Cheney. “Causally Consistent Dynamic Slicing”. In: *27th International Conference on Concurrency Theory (CONCUR)*. Vol. 59. LIPIcs. 2016, 18:1–18:15. doi: [10.4230/LIPIcs.CONCUR.2016.18](https://doi.org/10.4230/LIPIcs.CONCUR.2016.18).
- [23] Roly Perera et al. “Functional Programs that Explain Their Work”. In: *Proceedings of the 17th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 2012, pp. 365–376. doi: [10.1145/2364527.2364579](https://doi.org/10.1145/2364527.2364579).
- [24] Roly Perera et al. “Linked Visualisations via Galois Dependencies”. In: *Proceedings of the ACM on Programming Languages* 6.POPL (2022). doi: [10.1145/3498668](https://doi.org/10.1145/3498668).
- [25] Benjamin C. Pierce. *Types and Programming Languages*. 1st. The MIT Press, 2002. isbn: 0262162091.
- [26] Benjamin C. Pierce and David N. Turner. “Local type inference”. In: *ACM Transactions on Programming Languages and Systems* 22.1 (Jan. 2000), pp. 1–44. issn: 1558-4593. doi: [10.1145/345099.345100](https://doi.org/10.1145/345099.345100). url: <http://dx.doi.org/10.1145/345099.345100>.
- [27] Cecylia Rauszer. “Semi-Boolean algebras and their applications to intuitionistic logic with dual operations”. In: *Fundamenta Mathematicae* 83.3 (1974), pp. 219–249. doi: [10.4064/fm-83-3-219-249](https://doi.org/10.4064/fm-83-3-219-249).
- [28] Gonzalo E. Reyes and Houman Zolfaghari. “Bi-Heyting algebras, toposes and modalities”. In: *Journal of Philosophical Logic* 25.1 (1996), pp. 25–43. doi: [10.1007/BF00357841](https://doi.org/10.1007/BF00357841).
- [29] Wilmer Ricciotti et al. “Imperative Functional Programs that Explain Their Work”. In: *Proceedings of the ACM on Programming Languages* 1.ICFP (2017). doi: [10.1145/3110258](https://doi.org/10.1145/3110258).
- [30] Jeremy G. Siek and Walid Taha. “Gradual Typing for Functional Languages”. In: *Scheme and Functional Programming Workshop*. 2006, pp. 81–92.

- [31] Jeremy G. Siek et al. “Refined Criteria for Gradual Typing”. In: *1st Summit on Advances in Programming Languages (SNAPL 2015)*. Vol. 32. LIPIcs. 2015, pp. 274–293. doi: [10.4230/LIPIcs.SNAPL.2015.274](https://doi.org/10.4230/LIPIcs.SNAPL.2015.274).
- [32] Frank Tip. “A Survey of Program Slicing Techniques”. In: *Journal of Programming Languages* 3.3 (1995), pp. 121–189.
- [33] Sam Tobin-Hochstadt and Matthias Felleisen. “Logical Types for Untyped Languages”. In: *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 2010, pp. 117–128. doi: [10.1145/1863543.1863561](https://doi.org/10.1145/1863543.1863561).
- [34] Mitchell Wand. “Finding the Source of Type Errors”. In: *Proceedings of the 13th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 1986, pp. 38–43. doi: [10.1145/512644.512648](https://doi.org/10.1145/512644.512648).
- [35] Mark Weiser. “Program Slicing”. In: *IEEE Transactions on Software Engineering* SE-10.4 (1984), pp. 352–357. doi: [10.1109/TSE.1984.5010248](https://doi.org/10.1109/TSE.1984.5010248).
- [36] Baijun Wu and Sheng Chen. “How Type Errors Were Fixed and What Students Did?” In: *Proceedings of the ACM on Programming Languages* 1.OOPSLA (2017). doi: [10.1145/3133929](https://doi.org/10.1145/3133929).
- [37] Eric Zhao et al. “Total Type Error Localization and Recovery with Holes”. In: *Proceedings of the ACM on Programming Languages* 8.POPL (2024). doi: [10.1145/3632910](https://doi.org/10.1145/3632910).

A $\bigcirc$ Context Classification: Full Rule Set

Semantics/Statics/
CtxTyping,
Semantics/Marking/
CtxMarking

$\boxed{\Delta; \Gamma \vdash \mathcal{C} \text{ at } d \triangleright \Delta'; \Gamma' [m]}$ Context $\mathcal{C}$ at outer derivation mode d classifies its focus under $\Delta'; \Gamma'$ in focus mode m

$$\begin{array}{c}
s\bigcirc \frac{}{\Delta; \Gamma \vdash \bigcirc \text{ at } \uparrow \tau \triangleright \Delta; \Gamma [\uparrow \tau]} \quad \alpha\bigcirc \frac{}{\Delta; \Gamma \vdash \bigcirc \text{ at } \downarrow \tau \triangleright \Delta; \Gamma [\downarrow \tau]} \\
aSub \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \tau \sim \tau'}{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
s\lambda \frac{\Delta \vdash \tau_1 \text{ wf} \quad \Delta; \Gamma, x:\tau_1 \vdash \mathcal{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x:\tau_1. \mathcal{C} \text{ at } \uparrow \tau_1 \Rightarrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
a\lambda \frac{\tau \sim \tau_1 \Rightarrow \square \quad \Delta \vdash \tau_1 \text{ wf} \quad \Delta; \Gamma, x:\tau_1 \vdash \mathcal{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x:\tau_1. \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
a\lambda \Rightarrow \frac{\tau \sqcup \square \Rightarrow \square \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash \mathcal{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x. \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 27: Unmarked context classification: holes, subsumption, and lambdas.

$$\begin{array}{c}
s\circ_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square \Rightarrow \square \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash e \downarrow \tau_1}{\Delta; \Gamma \vdash \mathcal{C} \circ_1 e \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
s\circ_2 \frac{\Delta; \Gamma \vdash e \uparrow \tau \quad \tau \sqcup \square \Rightarrow \square \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash \mathcal{C} \text{ at } \downarrow \tau_1 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e \circ_2 \mathcal{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
s\langle \rangle_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \forall \cdot \square \equiv \forall \cdot \tau' \quad \Delta \vdash \sigma \text{ wf}}{\Delta; \Gamma \vdash \mathcal{C} \langle \sigma \rangle_1 \text{ at } \uparrow [0 \mapsto \sigma] \tau' \triangleright \Delta'; \Gamma' [m]} \\
s\pi_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_1 \mathcal{C} \text{ at } \uparrow \tau_1 \triangleright \Delta'; \Gamma' [m]} \\
s\pi_2 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_2 \mathcal{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
s\Lambda \frac{\Delta, \alpha; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \Lambda \alpha. \mathcal{C} \text{ at } \uparrow \forall \alpha. \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 28: Unmarked context classification: application, type application, projections, type abstraction.

$$\begin{array}{c}
s\&_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau_1 \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma \vdash e \uparrow \tau_2}{\Delta; \Gamma \vdash \mathcal{C} \&_1 e \text{ at } \uparrow \tau_1 \times \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
s\&_2 \frac{\Delta; \Gamma \vdash e \uparrow \tau_1 \quad \Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e \&_2 \mathcal{C} \text{ at } \uparrow \tau_1 \times \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
a\&_1 \frac{\tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2 \quad \Delta; \Gamma \vdash \mathcal{C} \text{ at } \downarrow \tau_1 \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma \vdash e \downarrow \tau_2}{\Delta; \Gamma \vdash \mathcal{C} \&_1 e \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
a\&_2 \frac{\tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2 \quad \Delta; \Gamma \vdash e \downarrow \tau_1 \quad \Delta; \Gamma \vdash \mathcal{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e \&_2 \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
sl_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_1 \mathcal{C} \text{ at } \uparrow \tau + \square \triangleright \Delta'; \Gamma' [m]} \\
sl_2 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_2 \mathcal{C} \text{ at } \uparrow \square + \tau \triangleright \Delta'; \Gamma' [m]} \\
al_1 \frac{\tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash \mathcal{C} \text{ at } \downarrow \tau_1 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_1 \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
al_2 \frac{\tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash \mathcal{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_2 \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 29: Unmarked context classification: pairs and injections.

$$\begin{array}{c}
scase_1 \frac{\Delta; \Gamma \vdash e \uparrow \tau \quad \Delta; \Gamma, x:\tau_1 \vdash \mathcal{C} \text{ at } \uparrow \tau'_1 \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, y:\tau_2 \vdash e' \uparrow \tau'_2 \quad \tau'_1 \sim \tau'_2}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. \mathcal{C} \mid y. e' \text{ at } \uparrow \tau'_1 \sqcup \tau'_2 \triangleright \Delta'; \Gamma' [m]} \\
scase_2 \frac{\Delta; \Gamma \vdash e \uparrow \tau \quad \Delta; \Gamma, x:\tau_1 \vdash e' \uparrow \tau'_1 \quad \Delta; \Gamma, y:\tau_2 \vdash \mathcal{C} \text{ at } \uparrow \tau'_2 \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \tau'_1 \sim \tau'_2}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e' \mid y. \mathcal{C} \text{ at } \uparrow \tau'_1 \sqcup \tau'_2 \triangleright \Delta'; \Gamma' [m]} \\
acase_1 \frac{\Delta; \Gamma \vdash e \uparrow \tau_0 \quad \Delta; \Gamma, x:\tau_1 \vdash \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau_0 \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, y:\tau_2 \vdash e' \downarrow \tau}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. \mathcal{C} \mid y. e' \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
acase_2 \frac{\Delta; \Gamma \vdash e \uparrow \tau_0 \quad \Delta; \Gamma, x:\tau_1 \vdash e' \downarrow \tau \quad \Delta; \Gamma, y:\tau_2 \vdash \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau_0 \sqcup \square + \square \equiv \tau_1 + \tau_2}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e' \mid y. \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 30: Unmarked context classification: case.

$$\begin{array}{c}
sdef_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, x:\tau' \vdash e \uparrow \tau}{\Delta; \Gamma \vdash \text{def } \mathcal{C} \vdash_1 e \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]} \\
sdef_2 \frac{\Delta; \Gamma \vdash e \uparrow \tau' \quad \Delta; \Gamma, x:\tau' \vdash \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \text{def } e \vdash_2 \mathcal{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]} \\
adef_1 \frac{\Delta; \Gamma \vdash \mathcal{C} \text{ at } \uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, x:\tau' \vdash e \downarrow \tau}{\Delta; \Gamma \vdash \text{def } \mathcal{C} \vdash_1 e \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
adef_2 \frac{\Delta; \Gamma \vdash e \uparrow \tau' \quad \Delta; \Gamma, x:\tau' \vdash \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \text{def } e \vdash_2 \mathcal{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 31: Unmarked context classification: let (def).

$\boxed{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } p \triangleright \Delta'; \Gamma' [m]}$ Marked context $C \multimap \check{C}$ at outer position p classifies its focus under $\Delta'; \Gamma'$ in focus mode m

$$\begin{array}{c}
ms\bigcirc \frac{}{\Delta; \Gamma \vdash \bigcirc \multimap \bigcirc \text{ at } \uparrow \tau \triangleright \Delta; \Gamma [\uparrow \tau]} \quad ma\bigcirc \frac{}{\Delta; \Gamma \vdash \bigcirc \multimap \bigcirc \text{ at } \downarrow \tau \triangleright \Delta; \Gamma [\downarrow \tau]} \\
\\
maSub \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \tau \sim \tau'}{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
maSub\uparrow \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \neg(\tau \sim \tau')}{\Delta; \Gamma \vdash C \multimap \langle \check{C} \rangle_{\tau' \not\sim \tau} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
ms\lambda \frac{\Delta \vdash \tau_1 \text{ wf} \quad \Delta; \Gamma, x:\tau_1 \vdash C \multimap \check{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x:\tau_1. C \multimap \lambda x:\tau_1. \check{C} \text{ at } \uparrow \tau_1 \Rightarrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
ma\lambda \frac{\tau \sim \tau_1 \Rightarrow \square \quad \tau \sqcup \tau_1 \Rightarrow \square \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta \vdash \tau_1 \text{ wf} \quad \Delta; \Gamma, x:\tau_1 \vdash C \multimap \check{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x:\tau_1. C \multimap \lambda x:\tau_1. \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 32: Marked context classification: holes, subsumption, annotated lambda.

$$\begin{array}{c}
ma\lambda \Rightarrow \frac{\tau \sqcup \square \Rightarrow \square \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash C \multimap \check{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x. C \multimap \lambda x. \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
ma\lambda \Rightarrow \uparrow \frac{\forall \tau_1, \tau_2. \tau \sqcup \square \Rightarrow \square \not\equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \square \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x. C \multimap \langle \check{C} \rangle_{\sim \Rightarrow} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
ms\lambda \Rightarrow \uparrow \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \square \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \lambda x. C \multimap \langle \check{C} \rangle_{\sim \Rightarrow} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]} \\
ms\circ_1 \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square \Rightarrow \square \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash e \multimap \check{e} \downarrow \tau_1}{\Delta; \Gamma \vdash C \circ_1 e \multimap \check{C} \circ_1 \check{e} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
ms\circ_1 \uparrow \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \forall \tau_1, \tau_2. \tau \sqcup \square \Rightarrow \square \not\equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash e \multimap \check{e} \downarrow \square}{\Delta; \Gamma \vdash C \circ_1 e \multimap \langle \check{C} \rangle_{\blacktriangleright \Rightarrow} \circ_1 \check{e} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]} \\
ms\circ_2 \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \tau \sqcup \square \Rightarrow \square \equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \tau_1 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e \circ_2 C \multimap \check{e} \circ_2 \check{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
ms\circ_2 \uparrow \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \forall \tau_1, \tau_2. \tau \sqcup \square \Rightarrow \square \not\equiv \tau_1 \Rightarrow \tau_2 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \square \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e \circ_2 C \multimap \langle \check{e} \rangle_{\blacktriangleright \Rightarrow} \circ_2 \check{C} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 33: Marked context classification: unannotated lambda and application.

$$\begin{array}{c}
\text{ms}_{<>_1} \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \forall \cdot \square \equiv \forall \cdot \tau' \quad \Delta \vdash \sigma \text{ wf}}{\Delta; \Gamma \vdash C \langle \sigma \rangle_1 \multimap \check{C} \langle \sigma \rangle_1 \text{ at } \uparrow [0 \mapsto \sigma] \tau' \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{<>_1} \uparrow \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \forall \tau'. \tau \sqcup \forall \cdot \square \not\equiv \forall \cdot \tau' \quad \Delta \vdash \sigma \text{ wf}}{\Delta; \Gamma \vdash C \langle \sigma \rangle_1 \multimap (\check{C})_{\blacktriangleright \forall \langle \sigma \rangle_1} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{\&_1} \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau_1 \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau_2}{\Delta; \Gamma \vdash C \&_1 e \multimap \check{C} \&_1 \check{e} \text{ at } \uparrow \tau_1 \times \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{\&_2} \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau_1 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e \&_2 C \multimap \check{e} \&_2 \check{C} \text{ at } \uparrow \tau_1 \times \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
\text{ma}_{\&_1} \frac{\tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \tau_1 \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma \vdash e \multimap \check{e} \downarrow \tau_2}{\Delta; \Gamma \vdash C \&_1 e \multimap \check{C} \&_1 \check{e} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{ma}_{\&_2} \frac{\tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2 \quad \Delta; \Gamma \vdash e \multimap \check{e} \downarrow \tau_1 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash e \&_2 C \multimap \check{e} \&_2 \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 34: Marked context classification: type application and pairs.

$$\begin{array}{c}
\text{ms}_{\iota_1} \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_1 C \multimap \iota_1 \check{C} \text{ at } \uparrow \tau + \square \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{\iota_2} \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_2 C \multimap \iota_2 \check{C} \text{ at } \uparrow \square + \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{ma}_{\iota_1} \frac{\tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \tau_1 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_1 C \multimap \iota_1 \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{ma}_{\iota_2} \frac{\tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \downarrow \tau_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \iota_2 C \multimap \iota_2 \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{\pi_1} \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_1 C \multimap \pi_1 \check{C} \text{ at } \uparrow \tau_1 \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{\pi_2} \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \tau \sqcup \square \times \square \equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_2 C \multimap \pi_2 \check{C} \text{ at } \uparrow \tau_2 \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{\pi_1} \uparrow \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \forall \tau_1, \tau_2. \tau \sqcup \square \times \square \not\equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_1 C \multimap \pi_1 (\check{C})_{\blacktriangleright \times} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}_{\pi_2} \uparrow \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m] \quad \forall \tau_1, \tau_2. \tau \sqcup \square \times \square \not\equiv \tau_1 \times \tau_2}{\Delta; \Gamma \vdash \pi_2 C \multimap \pi_2 (\check{C})_{\blacktriangleright \times} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 35: Marked context classification: injections and projections.

$$\begin{array}{c}
\text{mscase}_1 \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash C \multimap \check{C} \text{ at } \uparrow \tau'_1 \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, y:\tau_2 \vdash e' \multimap \check{e}' \uparrow \tau'_2 \quad \tau'_1 \sim \tau'_2}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. C \mid y. e' \multimap \text{case } \check{e} \text{ of } x. \check{C} \mid y. \check{e}' \text{ at } \uparrow \tau'_1 \sqcup \tau'_2 \triangleright \Delta'; \Gamma' [m]} \\
\text{mscase}_1 \uparrow \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \forall \tau_1, \tau_2. \tau \sqcup \square + \square \not\equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau'_1 \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma \vdash e' \multimap \check{e}' \uparrow \tau'_2}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. C \mid y. e' \multimap \text{case } (\check{e})_{\blacktriangleright+} \text{ of } x. \check{C} \mid y. \check{e}' \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]} \\
\text{mscase}_1 \nearrow \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash C \multimap \check{C} \text{ at } \uparrow \tau'_1 \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, y:\tau_2 \vdash e' \multimap \check{e}' \uparrow \tau'_2 \quad \neg(\tau'_1 \sim \tau'_2)}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. C \mid y. e' \multimap \text{case } \check{e} \text{ of } x. \check{C} \mid y. \check{e}' \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]} \\
\text{mscase}_2 \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash e' \multimap \check{e}' \uparrow \tau'_1 \quad \Delta; \Gamma, y:\tau_2 \vdash C \multimap \check{C} \text{ at } \uparrow \tau'_2 \triangleright \Delta'; \Gamma' [m] \quad \tau'_1 \sim \tau'_2}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e' \mid y. C \multimap \text{case } \check{e} \text{ of } x. \check{e}' \mid y. \check{C} \text{ at } \uparrow \tau'_1 \sqcup \tau'_2 \triangleright \Delta'; \Gamma' [m]} \\
\text{mscase}_2 \uparrow \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \forall \tau_1, \tau_2. \tau \sqcup \square + \square \not\equiv \tau_1 + \tau_2 \quad \Delta; \Gamma \vdash e' \multimap \check{e}' \uparrow \tau'_1 \quad \Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau'_2 \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e' \mid y. C \multimap \text{case } (\check{e})_{\blacktriangleright+} \text{ of } x. \check{e}' \mid y. \check{C} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]} \\
\text{mscase}_2 \nearrow \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau \quad \tau \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash e' \multimap \check{e}' \uparrow \tau'_1 \quad \Delta; \Gamma, y:\tau_2 \vdash C \multimap \check{C} \text{ at } \uparrow \tau'_2 \triangleright \Delta'; \Gamma' [m] \quad \neg(\tau'_1 \sim \tau'_2)}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e' \mid y. C \multimap \text{case } \check{e} \text{ of } x. \check{e}' \mid y. \check{C} \text{ at } \uparrow \square \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 36: Marked context classification: synthesis-mode case.

$$\begin{array}{c}
\text{macase}_1 \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau_0 \quad \tau_0 \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash C \multimap \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, y:\tau_2 \vdash e' \multimap \check{e}' \downarrow \tau}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. C \mid y. e' \multimap \text{case } \check{e} \text{ of } x. \check{C} \mid y. \check{e}' \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{macase}_1 \uparrow \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau_0 \quad \forall \tau_1, \tau_2. \tau_0 \sqcup \square + \square \not\equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\square \vdash C \multimap \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, y:\square \vdash e' \multimap \check{e}' \downarrow \tau}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. C \mid y. e' \multimap \text{case } (\check{e})_{\blacktriangleright+} \text{ of } x. \check{C} \mid y. \check{e}' \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{macase}_2 \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau_0 \quad \tau_0 \sqcup \square + \square \equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\tau_1 \vdash e' \multimap \check{e}' \downarrow \tau \quad \Delta; \Gamma, y:\tau_2 \vdash C \multimap \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e' \mid y. C \multimap \text{case } \check{e} \text{ of } x. \check{e}' \mid y. \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{macase}_2 \uparrow \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau_0 \quad \forall \tau_1, \tau_2. \tau_0 \sqcup \square + \square \not\equiv \tau_1 + \tau_2 \quad \Delta; \Gamma, x:\square \vdash e' \multimap \check{e}' \downarrow \tau \quad \Delta; \Gamma, y:\square \vdash C \multimap \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \text{case } e \text{ of } x. e' \mid y. C \multimap \text{case } (\check{e})_{\blacktriangleright+} \text{ of } x. \check{e}' \mid y. \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{msdef}_1 \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, x:\tau' \vdash e \multimap \check{e} \uparrow \tau}{\Delta; \Gamma \vdash \text{def } C \vdash_1 e \multimap \text{def } \check{C} \vdash_1 \check{e} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{msdef}_2 \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau' \quad \Delta; \Gamma, x:\tau' \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \text{def } e \vdash_2 C \multimap \text{def } \check{e} \vdash_2 \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 37: Marked context classification: analysis-mode case and let (synthesis).

$$\begin{array}{c}
\text{made}_f_1 \frac{\Delta; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau' \triangleright \Delta'; \Gamma' [m] \quad \Delta; \Gamma, x:\tau' \vdash e \multimap \check{e} \downarrow \tau}{\Delta; \Gamma \vdash \mathbf{def} C \vdash_1 e \multimap \mathbf{def} \check{C} \vdash_1 \check{e} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{made}_f_2 \frac{\Delta; \Gamma \vdash e \multimap \check{e} \uparrow \tau' \quad \Delta; \Gamma, x:\tau' \vdash C \multimap \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \mathbf{def} e \vdash_2 C \multimap \mathbf{def} \check{e} \vdash_2 \check{C} \text{ at } \downarrow \tau \triangleright \Delta'; \Gamma' [m]} \\
\text{ms}\Lambda \frac{\Delta, \alpha; \Gamma \vdash C \multimap \check{C} \text{ at } \uparrow \tau \triangleright \Delta'; \Gamma' [m]}{\Delta; \Gamma \vdash \Lambda \alpha. C \multimap \Lambda \alpha. \check{C} \text{ at } \uparrow \forall \alpha. \tau \triangleright \Delta'; \Gamma' [m]}
\end{array}$$

Figure 38: Marked context classification: let (analysis) and type abstraction.